



TOPIC-FOUR

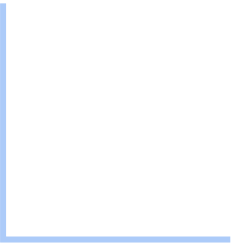
Docker Administration

Author: Arash Foroughi

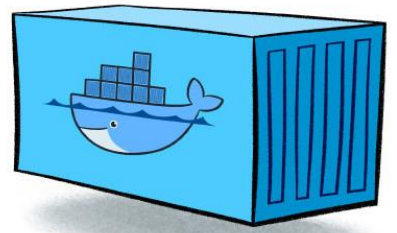




Docker Introduction



What is Containerization?



- It is the process of **encapsulating** software code along with all of its **dependencies** inside a single package so that it can be run **consistently** anywhere.

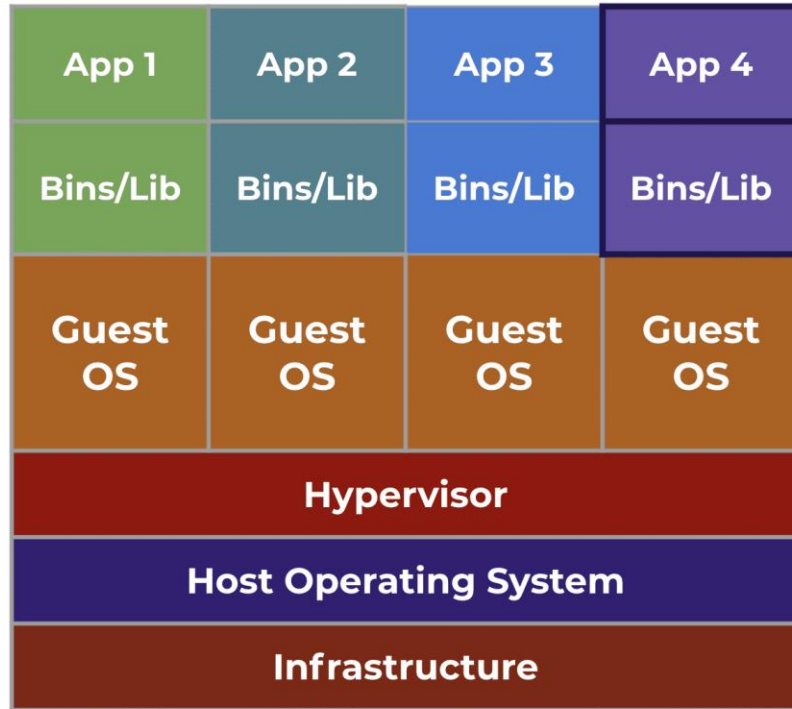


Introduction to Docker

- Docker is an **open source containerization** platform.
- It provides the ability to run applications in an **isolated environment (Containers)**.
- Containers are like **lightweight virtual machines** that can run directly on our host kernel **without need of a hypervisor**.
- Instead of virtualizing entire physical machine, containers virtualize **host OS only**.
- Containers share **host OS kernel, binaries & libraries** as well.



Types of Virtualization



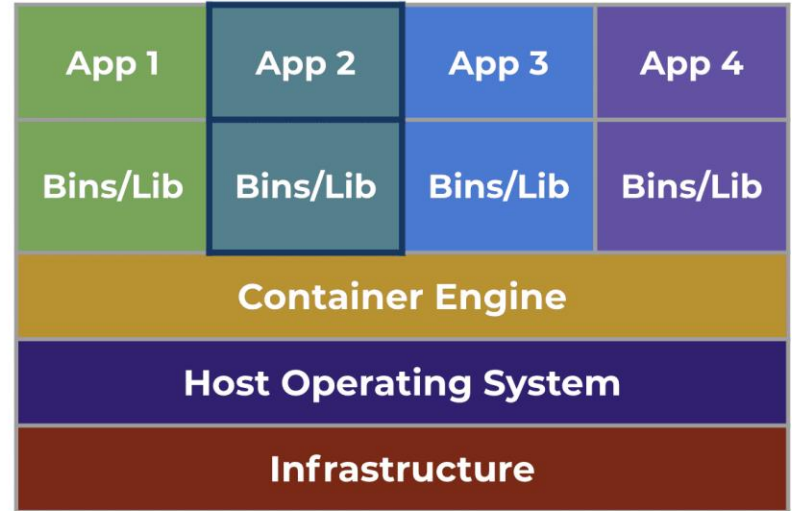
Virtualization

Virtual machine

Heavyweight hardware level virtualization

Container

Lightweight OS level virtualization



Containerization

NFV/SDN Architecture

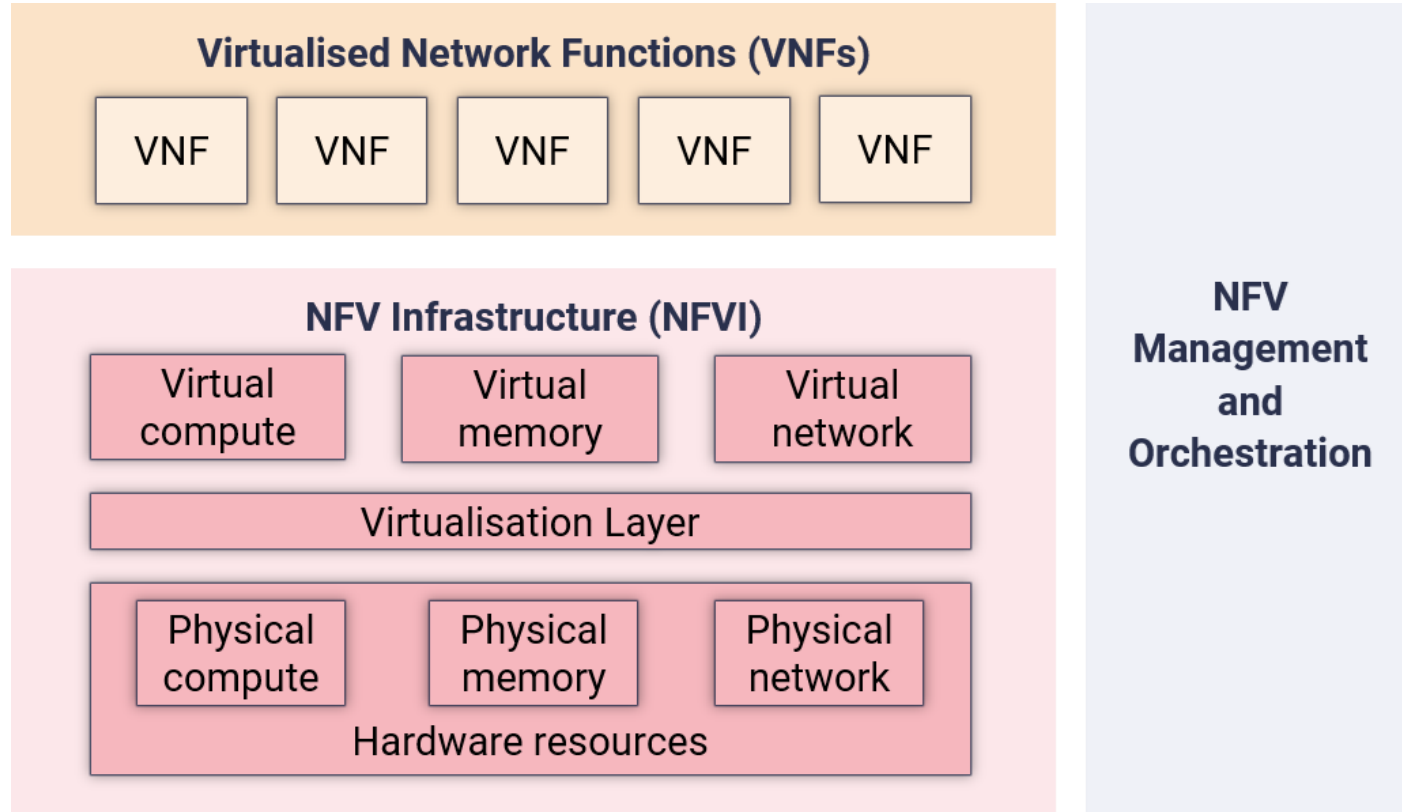
What is NFV Architecture?



- **NFV** stands for **Network Function Virtualization**.
- It is a technology that enables network functions, such as **firewalls**, **load balancers**, and **routers**, to be virtualized and run on standard hardware instead of dedicated appliances.
- This allows for **greater flexibility** and **scalability** in network infrastructure, as well as cost savings by reducing the need for specialized hardware.
- **NFV** is often used in conjunction with **software-defined networking (SDN)** to create more agile and efficient networks.

Network Functions Virtualization

NFV Architecture Design



What is SDN Architecture?



- **SDN** stands for **Software-Defined Networking**.
- It is an approach to **networking** that separates the **control plane** (which determines how data is forwarded) from the **data plane** (which actually forwards the data).
- This separation allows **system administrators** to manage and configure their networks more easily and efficiently, as they can control the entire network from a **central location** using software.
- **SDN** also enables **network automation**, which can help reduce errors and improve network performance.

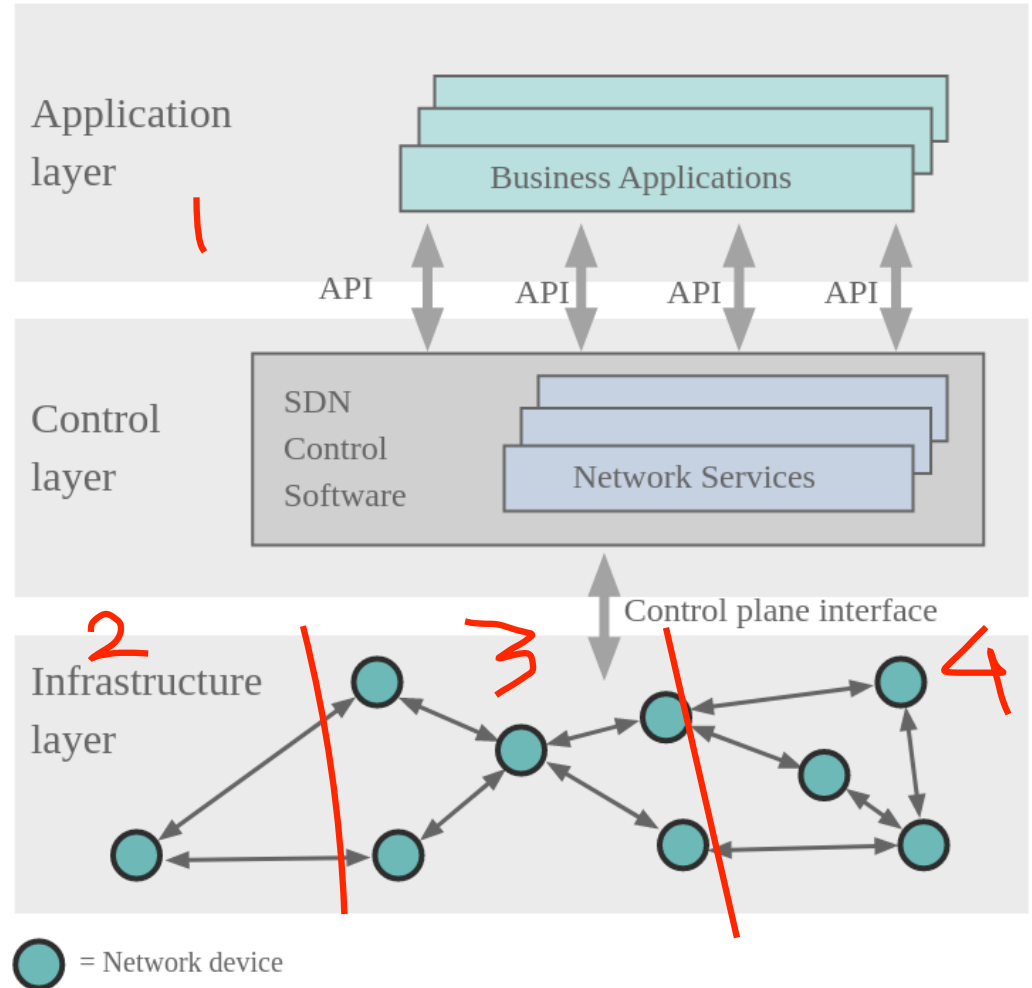
Software-Defined Networking

SDN Architecture

Docker Service
kubectrl

NFV/SDN Archit

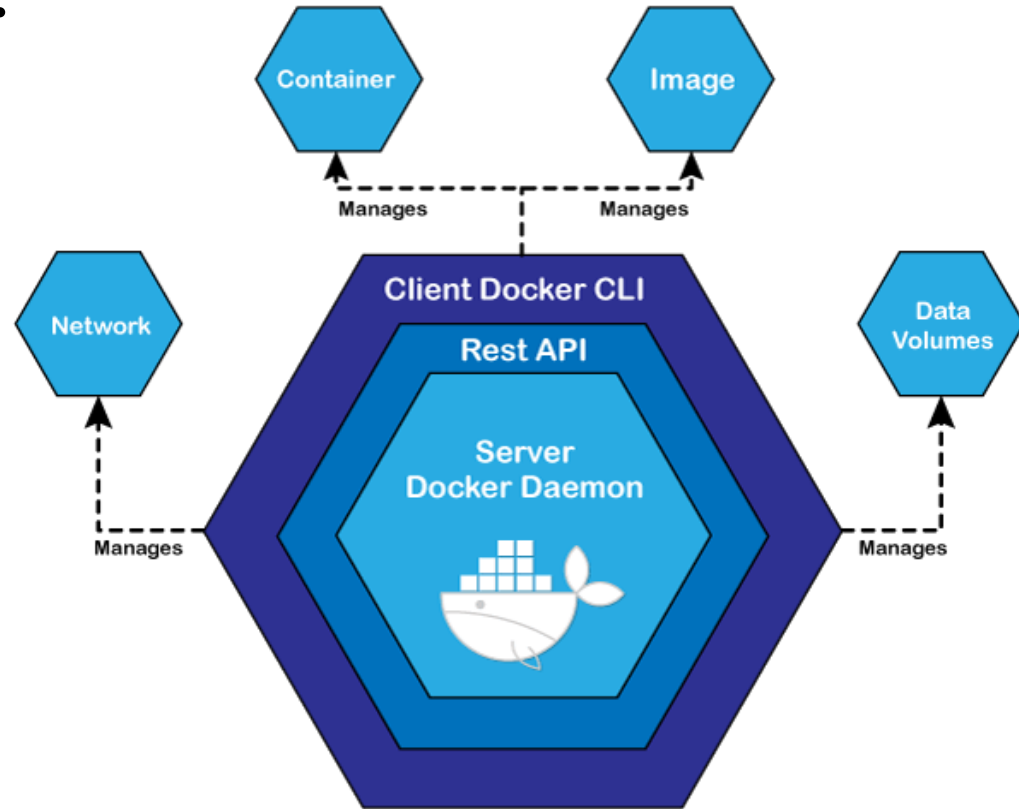
Docker swarm
Docker node
kubeadm



Docker Architecture

What is Docker Engine?

- Docker Engine is the **core of Docker**.
- It uses **server-client technology** to create & run containers.
- Major components of Docker Engine:
 - I. A Command-Line Interface (CLI).
 - II. A **REST API** which specifies interfaces that programs can use to talk to daemon & instruct it what to do.
 - III. A **server (Docker Daemon)** which is a type of long-running program called **dockerd** process.



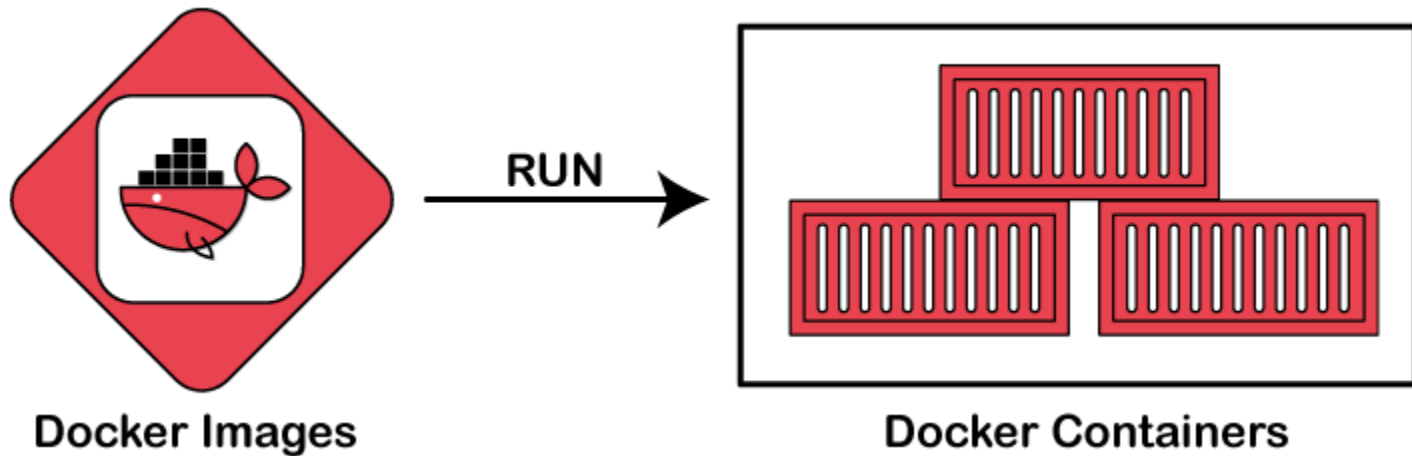
What is a Docker Image?

- A **Docker image** acts as a **set of instructions** to **build a Docker container**.
- An image is **comparable** to a **snapshot** in virtual machines.
- Images are created using **docker build** command.
- Docker allows you to **create** and **share software** using docker **images**.



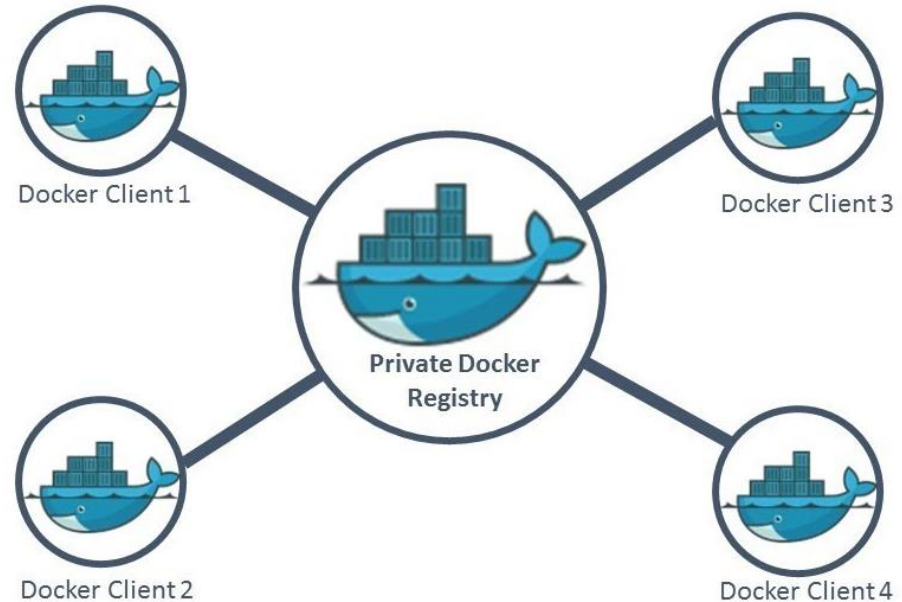
What is Docker Container?

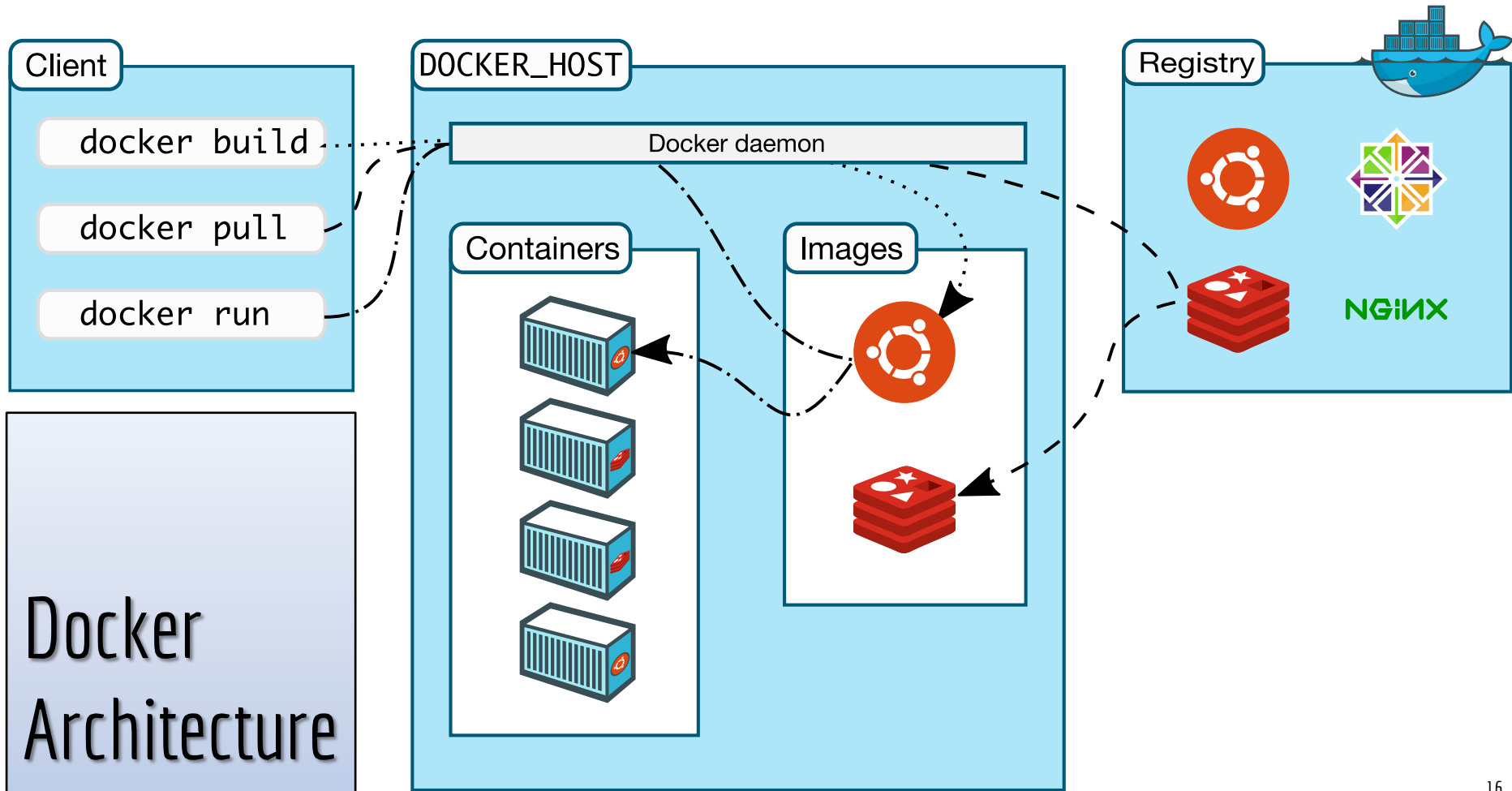
- **Docker Container** is a runnable instance of images.
- Through **Docker API** or **CLI**, we can **create** or **delete** a container.
- Containers are by default **isolated** from each other and their host machine.



What is Docker Registry?

- **Docker registry** is the place where **Docker images** are stored.
- **Docker Hub** is a **public registry** that anyone can access & configure images.
- With **docker run** or **docker pull** commands, can pull images from Registry.
- With **docker push** command, image will be pushed to the Registry.
- Registry can be on **public** or **local repository**.





Docker Installation

```
root@host:~# apt update; apt install ca-certificates curl gnupg -y

root@host:~# install -m 0755 -d /etc/apt/keyrings

root@host:~# curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --
    dearmor -o /etc/apt/keyrings/docker.gpg

root@host:~# chmod a+r /etc/apt/keyrings/docker.gpg

root@host:~# echo "deb [arch=$(dpkg --print-architecture) signed-
    by=/etc/apt/keyrings/docker.gpg]
    https://download.docker.com/linux/ubuntu $(. /etc/os-release &&
    echo "$VERSION_CODENAME") stable" | tee
    /etc/apt/sources.list.d/docker.list > /dev/null

root@host:~# apt update; apt install docker-ce docker-ce-cli containerd.io
    docker-buildx-plugin docker-compose-plugin -y
```

Run Our First Container on Docker

- ❑ Run a very simple container as the first container with `docker run` :

```
root@host:~# docker container run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
2db29710123e: Pull complete
Status: Downloaded newer image for hello-world:latest
```

Hello from Docker!

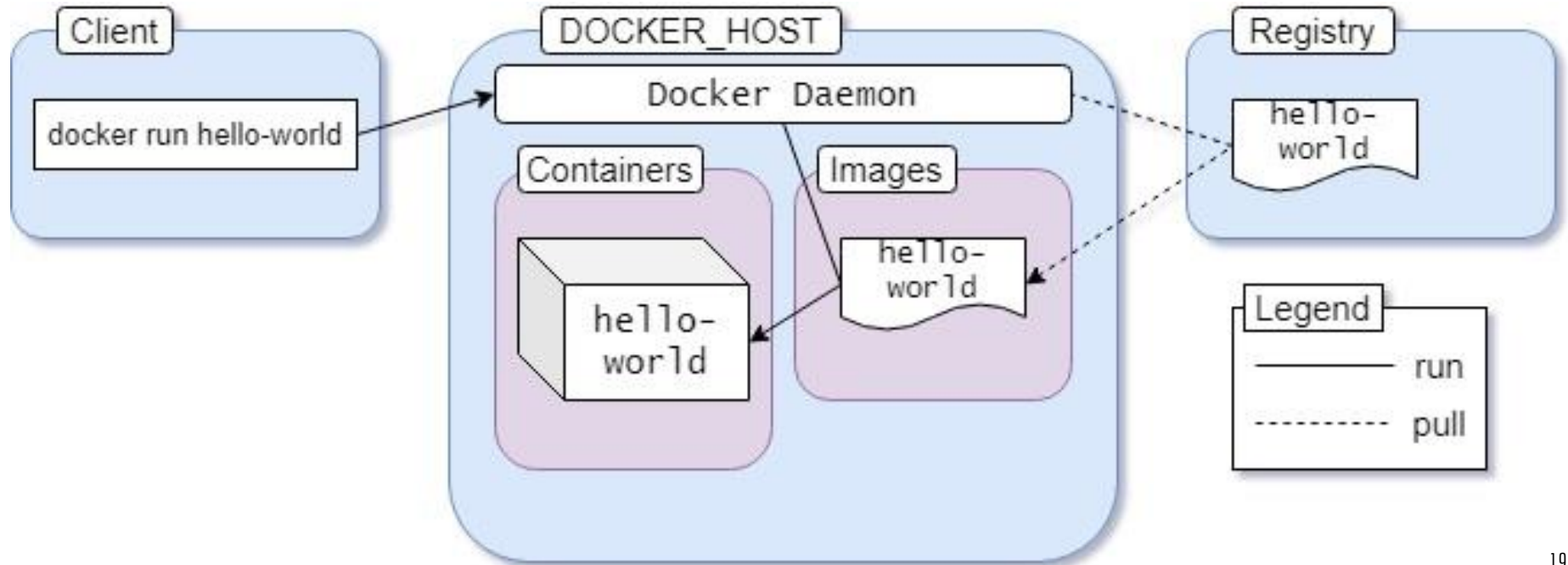
This message shows that your installation is working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to terminal.

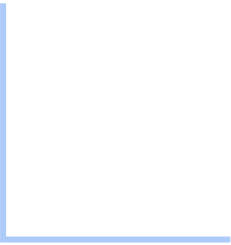
Run Our First Container on Docker

- Graphical process of running **hello-world** container using **docker**:





Manage Docker Containers



Login to Docker Hub



- Recently, Docker made a **rate limit** on **pulling request** during **every 6 hours**.
- **Authenticated users**, can have more pulling request in each 6 hours.
- So, first needs to **create a Docker Hub account** in <https://hub.docker.com/>.
- Then, **login to our Docker Hub** account from terminal.

```
root@host:~# docker login -u USERNAME -p PASSWORD
WARNING! Using --password via the CLI is insecure. Use --password-stdin.
WARNING! Your password will be stored unencrypted in
/root/.docker/config.json.
Configure a credential helper to remove this warning.

Login Succeeded
```

Run Containers

- ❑ Create & run a container using **docker run** :

```
root@host:~# docker run [IMAGE-NAME]
```

- ❑ Image name can be any image from Docker Hub or local machine.

docker run = docker create + docker start.

```
root@host:~# docker create hello-world
```

```
b2941618f37d26f04a16820c98c8a7a3d4240c6e4b4a16a134d493d931332a3d
```

```
root@host:~# docker start b2941618 -a
```

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly
```

- For using Docker with a regular user, it must be added to Docker group.

Listing Containers

❑ Listing the running containers using docker ps:

```
root@host:~# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

```
root@host:~# docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
B2941618f37d	hello-world	"/hello"	17 minutes	Exited 16 minutes ago		vigilant_curie
2095a6871a01	hello-world	"/hello"	17 minutes	Exited 17 minutes ago		lucid_edison

docker start [id] --> Start a container is not running.

docker restart [id] --> Restart running container or start stopped container.

docker rm [id] --> Remove history of a stopped container.

Listing Pulled Images

❏ Listing the pulled images locally using **docker image ls** :

```
root@host:~# docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
mynginx	latest	1d7555297a8d	5 minutes ago	132MB
nginx	latest	1ad8141779e0	7 minutes ago	132MB
<none>	<none>	c6621c2f7fa6	9 minutes ago	132MB
<none>	<none>	d11b91ed0928	10 minutes ago	132MB
ubuntu	latest	ba6acccedd29	12 days ago	72.8MB
redis	latest	7faaec683238	2 weeks ago	113MB
nginx	<none>	87a94228f133	2 weeks ago	133MB
hello-world	latest	feb5d9fea6a5	4 weeks ago	13.3kB

Running Containers in Interactive Mode

- ❑ Images can encapsulate an entire OS inside them, like **Ubuntu**, **Fedora**, & etc.
- ❑ For using **interactive** mode of a container like an OS, use **-it** option:

```
root@host:~# docker run -it ubuntu:latest
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
7b1a6ab2e44d: Pull complete
Digest:
sha256:626ffe58f6e7566e00254b638eb7e0f3b11d4da9675088f4781a50ae288f3322
Status: Downloaded newer image for ubuntu:latest
root@93736acd430d:/# exit                                --> Ctrl+P , Ctrl+Q
root@ubuntu:~# docker run ubuntu ls
root@ubuntu:~# docker run ubuntu apt update
```

Running Containers in Detached Mode

- ❑ **Detach mode** here means run containers in **background**, like a service.
- ❑ For run a container in **detach** mode, use **docker run -d [IMAGE]** :

```
root@host:~# docker run -dit redis:latest
```

```
5e7ec0cbad94e42657d4387fe8452ca6998a34180352bdcf95b4a99fb6ab028c
```

```
root@host:~# docker run -dit nginx:latest
```

```
9f828145cdd2d4a3f68a592c56d21b769bc97ba85e68d18e14d3ff98d9d79c41
```

```
root@host:~# docker ps
```

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
9f828145cdd2	nginx	"/docker-entrypoint...."	Up 2 minutes	80/tcp	admiring_gold
5e7ec0cbad94	redis	"docker-entrypoint.s..."	Up 13 minutes	6379/tcp	cool_colden

Connecting to Inside of a Container

- ❑ Executing commands inside a running container, using **docker exec** :

```
root@host:~# docker run -dit --name nginx1 nginx:latest
```

```
root@host:~# docker exec -it nginx1 cat /etc/hostname
```

```
9f828145cdd2
```

```
root@host:~# docker exec -it nginx1 /bin/bash
```

```
root@9f828145cdd2:/# cat /etc/hostname
```

```
9f828145cdd2
```

```
root@9f828145cdd2:/#
```

Stop/Kill a Running Container

- ❑ For stopping a container using **SIGTERM** signal, **docker stop [id]** .
- ❑ For killing container immediately using **SIGKILL** signal, **docker kill [id]** .

```
root@ubuntu:~# docker ps
```

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
9f828145cdd2	nginx	"/docker-entrypoint...."	Up 2 minutes	80/tcp	admiring_gold
5e7ec0cbad94	redis	"docker-entrypoint.s..."	Up 13 minutes	6379/tcp	cool_colden

```
root@ubuntu:~# docker stop admiring_gold
```

```
admiring_gold
```

```
root@ubuntu:~# docker kill 5e7ec0cbad94
```

```
5e7ec0cbad94
```

```
root@ubuntu:~# docker ps
```

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
--------------	-------	---------	--------	-------	-------

Importing/Exporting Containers

- ❑ Save an **instance** of a **running** or **paused** container to a **single tar file**.
- ❑ **Container's file system** can be exported using **docker export** as a tar file which it can be imported when required using **docker import** .

```
root@ubuntu:~# docker export determined_pascal > /root/my_nginx.tar
```

```
root@ubuntu:~# ls -l my_nginx.tar
```

```
-rw-r--r-- 1 root root 135770624 Oct 29 00:58 my_nginx.tar
```

```
root@ubuntu:~# docker import -m "message" my_nginx.tar my-nginx:v1
```

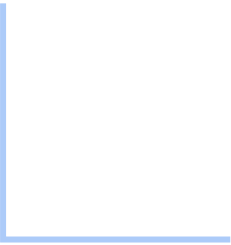
```
sha256:601c1156573fb3f0fae1dd963e85e6595e11ac662a2a189fc26923f2f152dc40
```

```
root@ubuntu:~# docker images my-nginx
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-nginx	v1	601c1156573f	10 seconds ago	132MB

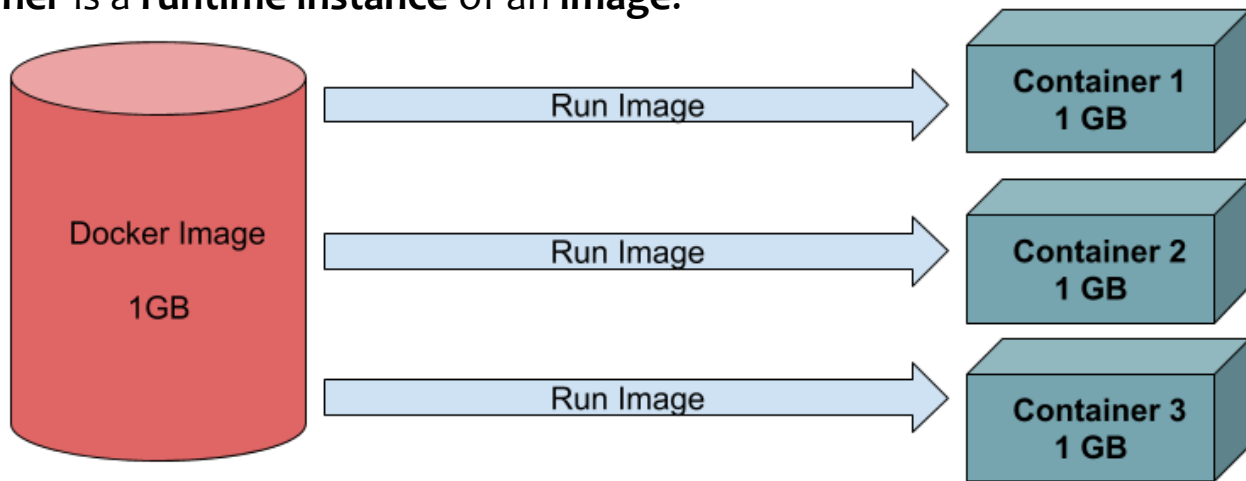


Docker Image Concepts

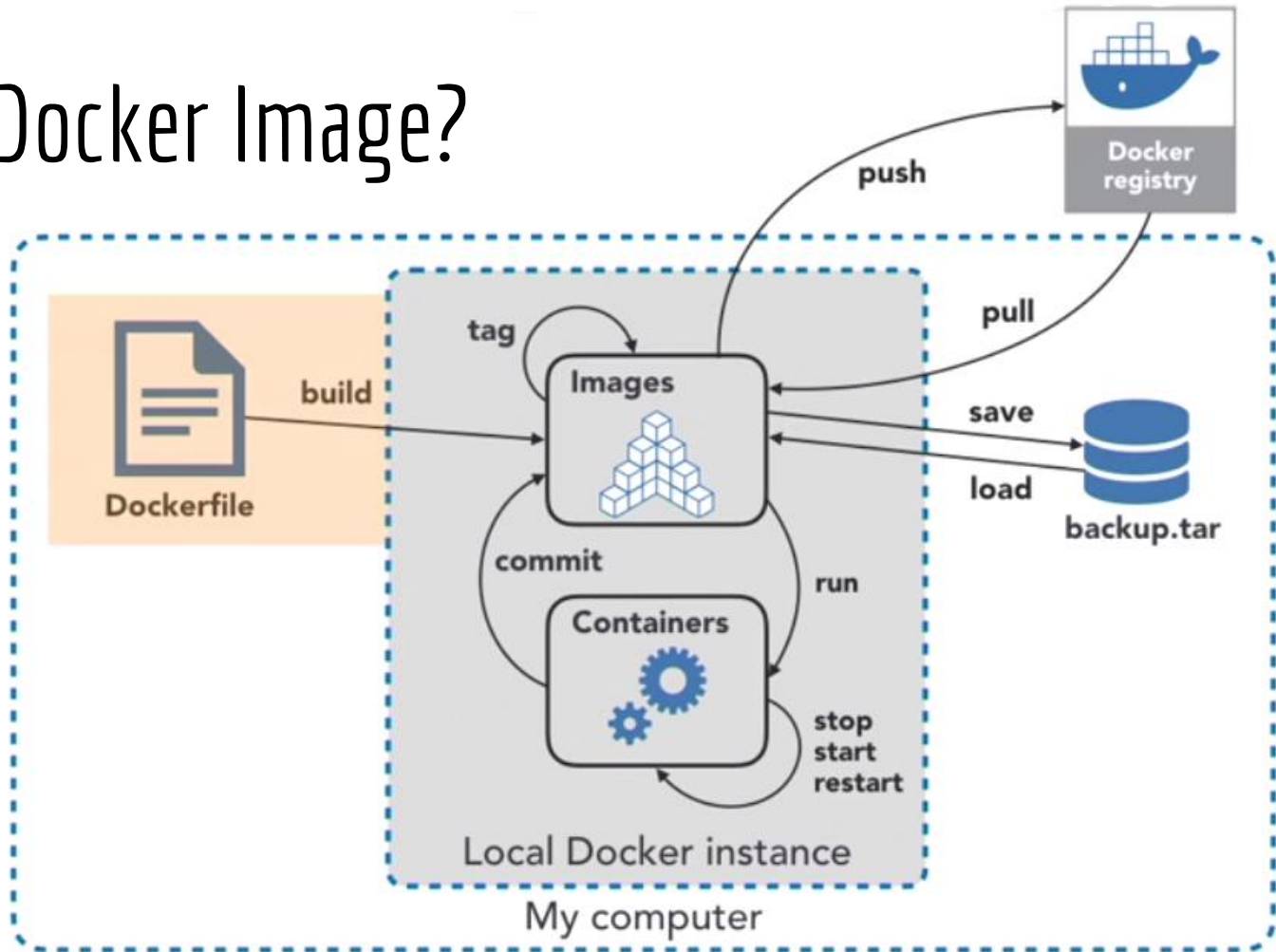


What's a Docker Image?

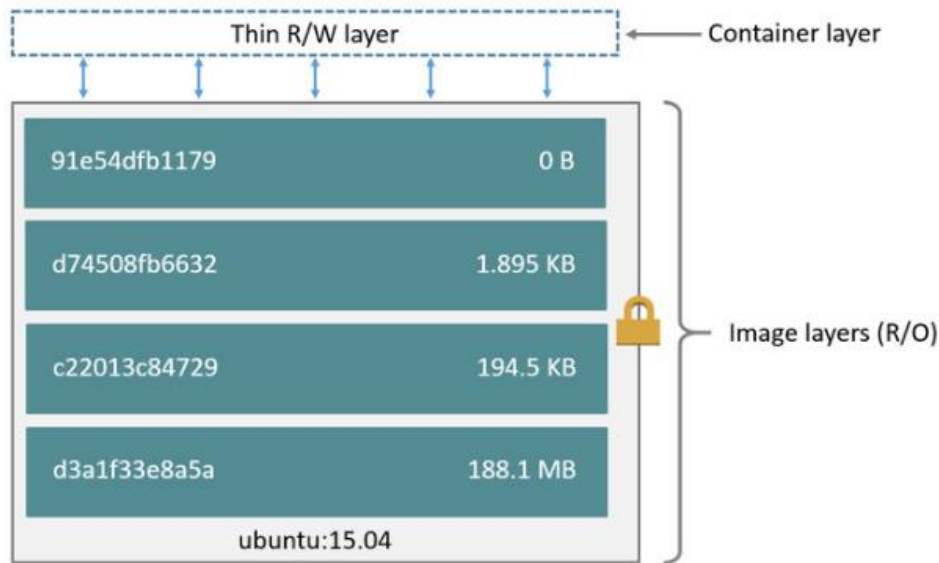
- **Image** is an **executable package** that includes everything needed **to run** an application.
- It contains **code, runtime libraries, environment variables & config files**.
- **Images** can exist **without containers**, whereas container needs to run an image to exist.
- A **container** is a **runtime instance** of an **image**.



What's a Docker Image?



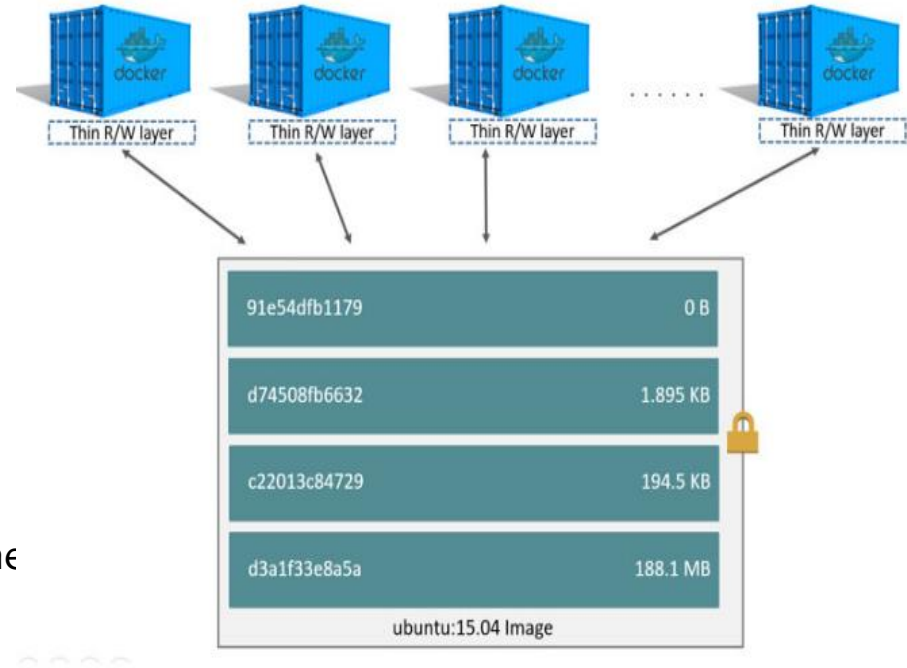
Docker Image Layers



- **Image Layers** are just **files** which created from **executing commands**.
- Layers are containing **files' changes** & they can be used **between different images**.
- In creating new image, doesn't need to **copy all older image files**, just need to **create a new layer** contain changes.
- There is one **upper layer (Container)** that it is **R/W**, other **lower layers** are **R/O**.

Image Layers & Containers

- From **each image**, hundreds of containers can be created.
- **Each container** has **specified upper layer** that it is Readable/Writable.
- With removing container, **just upper layer** removed & lower ones will remain.
- Because each container has its own **upper layer**, so, **multiple containers** can **share** one image with **its lower layers** together.

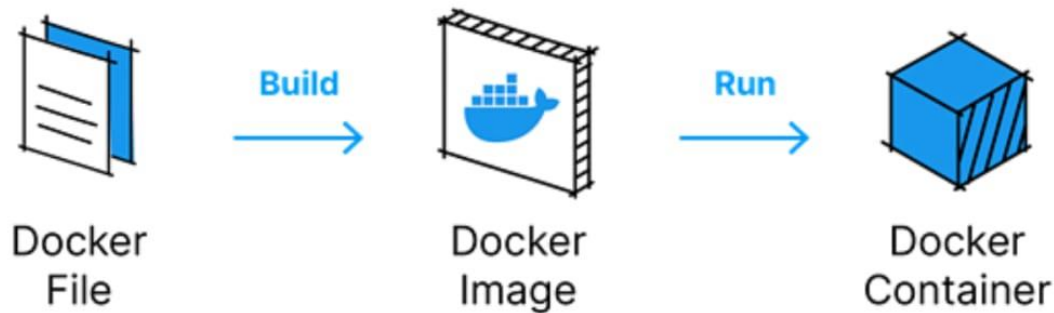


Location of Docker Images & Containers

- ❑ A **Docker container** consists of **network settings, volumes & images**.
- ❑ Location of Docker files depends on the OS & can be found by **docker info** :

```
root@host:~# docker info
Kernel Version: 5.11.0-16-generic
Operating System: Ubuntu 21.04
OSType: linux
CPUs: 4
Total Memory: 3.804GiB
Name: ubuntu
Docker Root Dir: /var/lib/docker
Debug Mode: false
Username: 920042231
Experimental: false
Insecure Registries:
  127.0.0.0/8
Live Restore Enabled: false
```

What is Dockerfile?



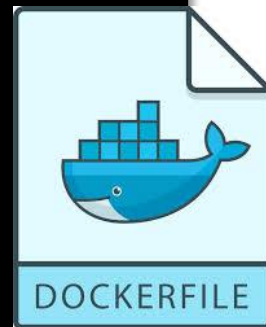
- Everyone can create **his own images** or use images **created by others**.
- For **build an image**, need to **make a Dockerfile** for creating an image & run it.
- Each instruction in a Dockerfile **creates a layer in the image**.
- With changing Dockerfile and **rebuilding image**, only **changed layers** are rebuilt.
- It will make images so **lightweight, small & fast**.
- A Dockerfile executed by **docker build**.

A Dockerfile Sample

- ❑ Here is a sample of **simple DockerFile**:

```
root@host:~# vi Dockerfile
FROM ubuntu:16.04
LABEL Description="test image" Vendor="Example Company"
RUN apt-get update
RUN apt-get install -y nginx openssh-server
```

- ❑ There are some **directives** in next page about **DockerFile**.



Instruction	Description
FROM	It will set the base image of the container.
LABEL	It is a key-value pair used to specify metadata information of the image.
MAINTAINER	It will give the detail of the author who created this Docker image.
RUN	It is used to execute the command on the base image and it will create a new layer .
COPY	It is used to copy local files to the container.
ADD	It works same as copy but having some more feature like we can extract local tar, add remote URL & add hidden files .
ENV	It is used to set environment variables in the key-value pair . These variables are set during the image build and are available after container created.
ARGS	It is used to set variables but it just set during the image build not after container created .
EXPOSE	It will expose the port to access the container. Container will listen on this network port.
VOLUME	It will creates a mount point with the specified name.
USER	It will sets the user name and user group to use when b the image.
WORKDIR	It will set the working directory . It will create the directory if not present.
CMD	It is used to set a command to execute first when the container starts .
ENTRYPOINT	It is used to set the main command for the image. It works as same as CMD instruction. The only difference between CMD and ENTRYPOINT is instructions are not overwritten in ENTRYPOINT.

Sample-1--Apache on Ubuntu

```
root@host:~# vi Dockerfile
# Get the base image
FROM ubuntu:16.04
# Install all packages
RUN apt update && apt -y upgrade && apt install apache2 elinks -y
# adding some content for Apache server
RUN echo "This is a test docker" > /var/www/html/index.html
# Copying setting file & adding some content to be served by apache
COPY /home/httpd.conf /etc/apache2/httpd.conf
# Defining a command to be run after the docker is up
ENTRYPOINT ["elinks"]
CMD ["localhost"]
```

Sample-2--Apache on CentOS

```
root@host:~# vi Dockerfile
FROM centos:7
MAINTAINER ArashForoughi
LABEL Remarks="This is a dockerfile example for Centos system"
RUN yum -y update && yum -y install httpd && yum clean all
COPY data/httpd.conf /etc/httpd/conf/httpd.conf
COPY /data/html.tar.gz /var/www/html/
EXPOSE 80
ENV HOME /root
WORKDIR /root
USER app
CMD ["ping" "127.0.0.1" ">>" "/dev/null"]
```

Sample-3

Redis on

Ubuntu

```
root@host:~# vi Dockerfile
FROM ubuntu:22.04
MAINTAINER Arash Foroughi arashforoughi@outlook.com

ENV REDIS_VERSION=4.0.9 \
    REDIS_USER=redis \
    REDIS_DATA_DIR=/var/lib/redis \
    REDIS_LOG_DIR=/var/log/redis

RUN apt update \
    && apt install -y redis-server=5:${REDIS_VERSION}* \
    && sed 's/^bind /# bind /' -i /etc/redis/redis.conf \
    && sed 's/^logfile /# logfile /' -i /etc/redis/redis.conf \
    && sed 's/^daemonize yes/daemonize no/' -i /etc/redis/redis.conf \
    && sed 's/^# unixsocket /unixsocket /' -i /etc/redis/redis.conf \
    && rm -rf /var/lib/apt/lists/*

COPY entrypoint.sh /sbin/entrypoint.sh
RUN chmod 755 /sbin/entrypoint.sh

EXPOSE 6379/tcp
ENTRYPOINT ["/sbin/entrypoint.sh"]
```

Sample-4 NodeJS App on Ubuntu

```
root@host:~# vi Dockerfile
```

```
FROM node:18.13.0-slim
```

```
RUN apt update && \  
apt install git curl iputils-ping -y && \  
apt clean all
```

```
WORKDIR /app
```

```
ADD . .
```

```
RUN chmod 777 /app/entrypoint.sh
```

```
RUN npm install
```

```
RUN npm install -g pm2
```

```
RUN npm install index.js
```

```
EXPOSE 9001
```

```
CMD ["/app/entrypoint.sh"]
```

Sample-5

Python

with pip

```
root@host:~# vi Dockerfile
FROM python:3.8-slim-buster AS runtime-image
RUN apt-get update && apt-get install -y --no-install-recommends git cron
libcurl4-openssl-dev libssl-dev build-essential
COPY docker/crontab /etc/cron.d/crontab
RUN chmod 0644 /etc/cron.d/crontab
RUN /usr/bin/crontab /etc/cron.d/crontab
ARG dest=/usr/src/app
# Create the application directories
RUN mkdir $dest
WORKDIR $dest
RUN mkdir ./src
RUN mkdir /data
# Copy the rest of the codebase into the image
COPY src/ ./src
COPY index.py .
COPY docker/requirements.txt /tmp/
RUN pip install -r /tmp/requirements.txt
## set environment variables
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
# Setup entrypoint.sh
COPY docker/docker-entrypoint.sh /
RUN chmod +x /docker-entrypoint.sh
ENTRYPOINT ["/docker-entrypoint.sh"]
```

Sample-6 NodeJS Dockerfile Multi Stage

```
root@host:~# vi Dockerfile
```

```
FROM node:16 AS build
ENV PATH /app/node_modules/.bin:$PATH
WORKDIR /app
ADD . ./
COPY package*.json ./
COPY *.tgz ./
RUN npm install --force
RUN npm install -g env-cmd
RUN rm -r ./node_modules/jest
RUN npm run build:prod

FROM nginx:1.19
COPY --from=build /app/build /usr/share/nginx/html
COPY ./nginx/nginx.conf /usr/share/nginx/nginx.conf
COPY ./localhost-cert.pem /usr/share/nginx/localhost-cert.pem
COPY ./localhost-key.pem /usr/share/nginx/localhost-key.pem
EXPOSE 80
EXPOSE 443
CMD ["nginx", "-c", "/usr/share/nginx/nginx.conf"]
```

Sample-7

PHP

Dockerfile

```
root@host:~# vi Dockerfile
FROM php:8.1-fpm
WORKDIR /var/www
RUN apt-get update && apt-get install -y build-essential curl \
    git gifsicle jpegoptim libfreetype6-dev libjpeg62-turbo-dev \
    libonig-dev libpng-dev libzip-dev locales optipng pngquant \
    unzip vim nodejs yarn && rm -rf /var/lib/apt/lists/*

# Install Node.js and Yarn
RUN curl -fsSL https://deb.nodesource.com/setup_16.x | bash - \
    && curl -sS https://dl.yarnpkg.com/debian/pubkey.gpg | apt-key add - \
    && echo "deb https://dl.yarnpkg.com/debian/ stable main" | \
    tee /etc/apt/sources.list.d/yarn.list

# Install PHP extensions
RUN docker-php-ext-install pdo_mysql mbstring zip exif pcntl \
    && docker-php-ext-configure gd --with-freetype --with-jpeg \
    && docker-php-ext-install gd

# Install Composer
RUN curl -sS https://getcomposer.org/installer | php -- --install-
dir=/usr/local/bin --filename=composer

# Start php-fpm server
CMD ["php-fpm"]
```

Build an Image from DockerFile

- ❑ To build an image by DockerFile, use **docker build** :
- ❑ Build command needs **Build's Context** which it's the path of DockerFile.

```
root@host:~# docker build .
```

```
  Sending build context to Docker daemon  36.86kB
```

```
Step 1/3 : FROM ubuntu
```

```
----> ba6acccedd29
```

```
Step 2/3 : LABEL Description="test image" Vendor="ACME Products"
```

```
----> Running in fb4ccf1c8b9d
```

```
Removing intermediate container fb4ccf1c8b9d
```

```
----> 5c8066a94c39
```

```
Step 3/3 : RUN apt-get update && apt-get install -y nginx openssh-server
```

```
----> Running in 2d9c9fdc0eb1
```

```
. . .
```

```
----> 57d04f47ea75
```

```
Successfully built cd2d97fc181d
```

Getting Build History

- ❑ Get **information** about the image use **docker image inspect [IMAGE-ID]**.
- ❑ To **see layers** which image includes, use **docker image history [IMAGE-ID]**.

```
root@host:~# docker image history cd2d97fc181d
```

IMAGE	CREATED	CREATED BY	SIZE
cd2d97fc181d	7 minutes ago	/bin/sh -c apt-get update && apt-get install...	168MB
d7c38a250ff6	9 minutes ago	/bin/sh -c #(nop) LABEL Description=test im...	0B
ba6acccedd29	4 weeks ago	/bin/sh -c #(nop) CMD ["bash"]	0B
<missing>	4 weeks ago	/bin/sh -c #(nop) ADD file:5d68d27cc15a80653...	72.8MB

Tagging an Image

❑ Image **tags** are **aliases** to the **image ID** like **cd2d97fc181d**.

```
root@host:~# docker tag c87d321cfffc9 test:V1.0
```

```
root@host:~# docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
test	V1.0	c87d321cfffc9	46 minutes ago	72.8MB

```
root@host:~# docker tag test:V1.0 test:V2.0
```

```
root@host:~# docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
test	V1.0	c87d321cfffc9	46 minutes ago	72.8MB
test	V2.0	c87d321cfffc9	46 minutes ago	72.8MB

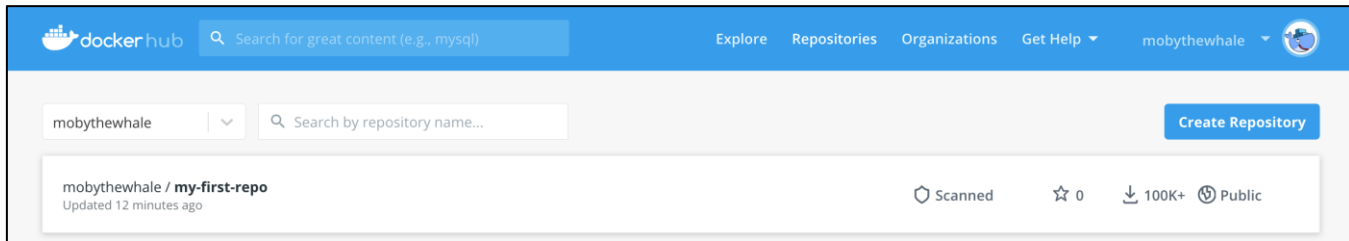
Build an Image with Tagging

❏ To build an image and add a tag on it, use **docker build -t [TAG] . :**

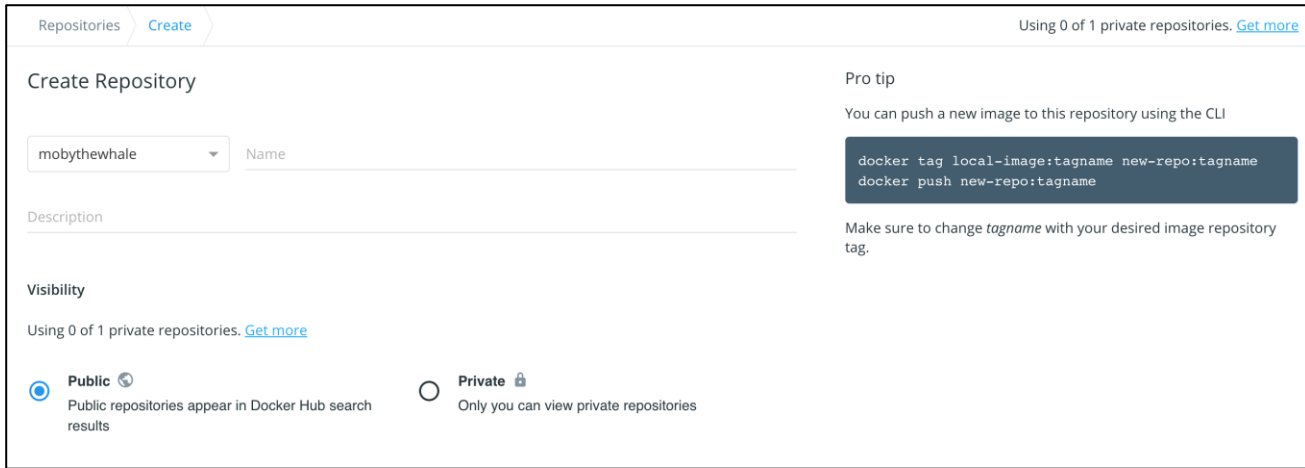
```
root@host:~# docker build --tag testapp:V2.6 .
Sending build context to Docker daemon 36.86kB
Step 1/3 : FROM ubuntu
----> ba6acccedd29
Step 2/3 : LABEL Description="test image" Vendor="ACME Products"
----> Running in fb4ccf1c8b9d
Removing intermediate container fb4ccf1c8b9d
----> 5c8066a94c39
Step 3/3 : RUN apt-get update && apt-get install -y nginx openssh-server
----> Running in 2d9c9fdc0eb1
. . . . .
----> 3ed52de4c92b
Successfully built 3ed52de4c92b
Successfully tagged testapp:V2.6
```

Pushing an Image to Docker Hub Registry

- First needs to **create a repository** in Docker Hub Registry.



1



2

Pushing an Image to Docker Hub Registry

- Second, **rename** local image to the **registry** designed with one of below ways:
 - `docker build -t <hub-user>/<repo-name>:<tag>`
 - `docker tag <existing-image> <hub-user>/<repo-name>:<tag>`
 - `docker commit <existing-container> <hub-user>/<repo-name>:<tag>`
- Third, push the new tagged image to the repository using `docker push`.

```
root@host:~# docker tag test:latest 9354165450/testrepo:test
```

```
root@host:~# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
test	latest	72b31b79d54d	7 minutes ago	241MB
9354165450/testrepo	test	72b31b79d54d	7 minutes ago	241MB

```
root@host:~# docker push 9354165450/testrepo:test
```

```
The push refers to repository [docker.io/920042231/testrepo]
```

```
38cdd4a90702: Pushed
```

```
test: digest: sha256:36830400109c62dd8d6cfd8927bc8069e09d59c size: 741
```

Pulling an Image from Registry

❑ To download a specific image or images, you can use **docker pull** :

```
root@host:~# docker pull centos:latest
Using default tag: latest
latest: Pulling from library/centos
a1d0c753277: Pull complete
Digest: sha256:a27fd8080b517143cbbbab9d34bbdee55bd6c473f432b177
Status: Download newer image for centos:latest
docker.io/library/centos:latest

root@host:~# docker pull 9354165450/testrepo:test
test: Pulling from 9354165450/testrepo
Digest: sha256:36830400109c6cfd892c8191f0f97ced111814e8f209d59c
Status: Downloaded newer image for 920042231/testrepo:test
docker.io/9354165450/testrepo:test
```

Removing an Image

❏ To remove a specific image or images, you can use **docker rmi** :

```
root@host:~# docker rmi debian
```

```
Untagged: debian:latest
```

```
Untagged: debian@sha256:4d6ab716de46badd7478847ec7a18f66027d0f8a329a43c
```

```
Deleted: sha256:f776cfb21b5e06bb5b4883eb15c11476b8275293c7f96d09b90f7f9
```

```
Deleted: sha256:6719d2d37fff5670ed40de690095743172de1b4434cb019b56f30b4
```

```
root@host:~# docker rmi --force hello-world
```

```
Untagged: hello-world:latest
```

```
Untagged: hello-world@sha256:cc15c9f1804f3a725c8d05e158653a563f15e4f685
```

```
Deleted: sha256:feb5d9fea6a5e9606aa995e825965ba48de054caab5ef356dc6b412
```

What are Dangling Images?

- **Dangling images** are layers that have **no relationship** to any **tagged images**.
- They no longer serve a purpose and **consume disk space**.
- When an image created with **Dockerfile without tagging**, it will be saved as dangling.

❑ **docker image prune** will remove all unused **dangling** images.

```
root@host:~# docker images --filter dangling=true  
git version 2.30.2
```

```
root@host:~# docker image prune  
WARNING! This will remove all dangling images.  
Are you sure you want to continue? [y/N] y  
Deleted Images:  
deleted: sha256:645acbd6ead13b24c5924a97b69dfe689f063aa5bf25af5528499f  
deleted: sha256:4eacf93363243b49bd9816c2733c45b988dd69ba78f41117665cb0  
deleted: sha256:581f7fb900fee1f420d6857c0263524909bfa82e5f1d54e93cead2  
Total reclaimed space: 168MB
```

Searching for an Image

- ❑ For searching in registry for a specific image, use **docker search** command:

```
root@host:~# docker search fedora
```

NAME	DESCRIPTION	STARS	OFFICIAL	AUTOMATED
fedora	Official Docker builds of Fedora	1013	[OK]	
fedora/apache		35		[OK]
arm32v7/fedora	Official Docker builds of Fedora	8		
darksheer/fedora22	Fedora 22 Image -- Updated hourly	3		[OK]
arm64v8/fedora	Official Docker builds of Fedora	2		[OK]

- **STARS** : How many people have liked the image.
- **OFFICIAL** : Specifies whether the image is built from a trusted source.
- **AUTOMATED** : Tells whether the images is built automatically with a push in GitHub or Bitbucket.

```
root@host:~# docker search --filter stars=100 --filter is-official=true
```

NAME	DESCRIPTION	STARS	OFFICIAL	AUTOMATED
fedora	Official Docker builds of Fedora	1013	[OK]	

Saving/Loading Images

- **Docker save/load:** works with **images**, it saves a **non-running image** to a file.
- **Docker export/import:** works with **containers**, it exports **running instances** to a file.

```
root@host:~# vi Dockerfile
FROM busybox
CMD echo $((40 + 2))

root@host:~# docker build --tag testapp:V1.0 .
root@host:~# docker image ls
  REPOSITORY   TAG       IMAGE ID        CREATED         SIZE
  testapp      V1.0      3f112a117e18    6 seconds ago  1.24MB
  busybox      latest    7138284460ff    3 days ago     1.24MB

root@host:~# docker save testapp > ~/testapp.tar
root@host:~# docker load --input ~/testapp.tar
Loaded image: testapp:V1.0

Or root@host:~# docker import ~/testapp.tar testapp:V1.0
sha256:42fc8fd7a31462d9a278713a674a449813329957e1c8255dcee0df5dc24d
```

Committing Changes to an Image

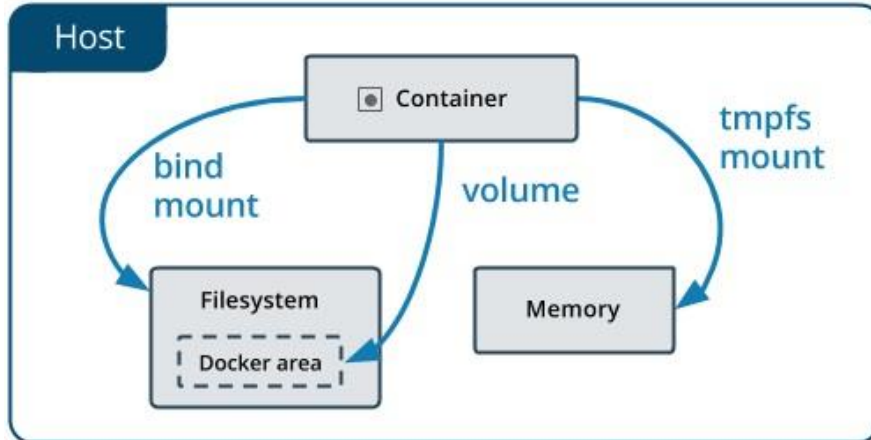
- ❑ Every changes on images/containers can be commit as a new version.

```
root@host:~# docker run -d --name mynginx nginx
root@host:~# docker exec -it mynginx bash
root@920e5cd499cc:/# cat > /usr/share/nginx/html/index.html
root@920e5cd499cc:/# Ctrl+p then Ctrl+q
root@host:~# docker ps
CONTAINER ID  IMAGE  COMMAND                  CREATED   STATUS    PORTS    NAMES
7cd054fc8223  nginx  "/entrypoint.sh"         1 minute 1 minute  80/tcp   mynginx
root@host:~# docker commit mynginx mynginx2
sha256:2add2b16d4e96c3370a0ebe197e6d861fa43d79a198adae3b278d7a061dd78b2
root@host:~# docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
mynginx2      latest   2add2b16d4e9   4 seconds ago  145MB
nginx         latest   04661cdce581   5 days ago    141MB
```

Docker Volumes

Manage Data in Docker

- Applications on docker containers **need data** & **data need to be stored** some where.
 - Docker can store container data on **Host Hard Disk** or **Memory**:
- 1) **Volumes:** Recommended way to persist data, stored at `/var/lib/docker/volumes/`.
 - 2) **Bind Mounts:** have limited functionality & exact file path on the host must use.
 - 3) **tmpfs mounts:** Stored only in a host's memory in Linux (least recommended)



Docker Storage Drivers

- **Docker Storage Driver** controls how images/containers are **stored & managed** on host.
- **Docker Engine** provides following **storage drivers** on Linux:

Driver	Description
overlay2	It is preferred storage driver for all currently Linux distros & requires no extra configs.
btrfs & zfs	These storage drivers has advanced options, like “ snapshots ”, but require more configs.
vfs	It is used for testing purposes. Its performance is poor & not recommended for live use.
aufs	It was preferred storage driver for Docker 18.06 & older on Ubuntu 14.04 & kernel 3.13 which had no support for overlay2.
devicemapper	Devicemapper requires direct-lvm environment. It was preferred storage driver for CentOS & RHEL, as their kernel version did not support overlay2.
overlay	The legacy overlay driver was used for kernels that did not support the “multiple-lowerdir” feature required for overlay2.

Supported Storage Drivers per Linux Distro.

- The **operating system** and **kernel** may not support **every storage driver**.

Linux Distro.	Recommended Storage Drivers	Alternate Drivers
Ubuntu	overlay2	overlay, devicemapper, aufs, zfs, vfs
Debian	overlay2	overlay, devicemapper, aufs, vfs
CentOS	overlay2	overlay, devicemapper , zfs, vfs
Fedora	overlay2	overlay, devicemapper, zfs, vfs
SLES 15	overlay2	overlay, devicemapper, vfs
RHEL	overlay2	overlay, devicemapper, vfs

Supported Backing Filesystems

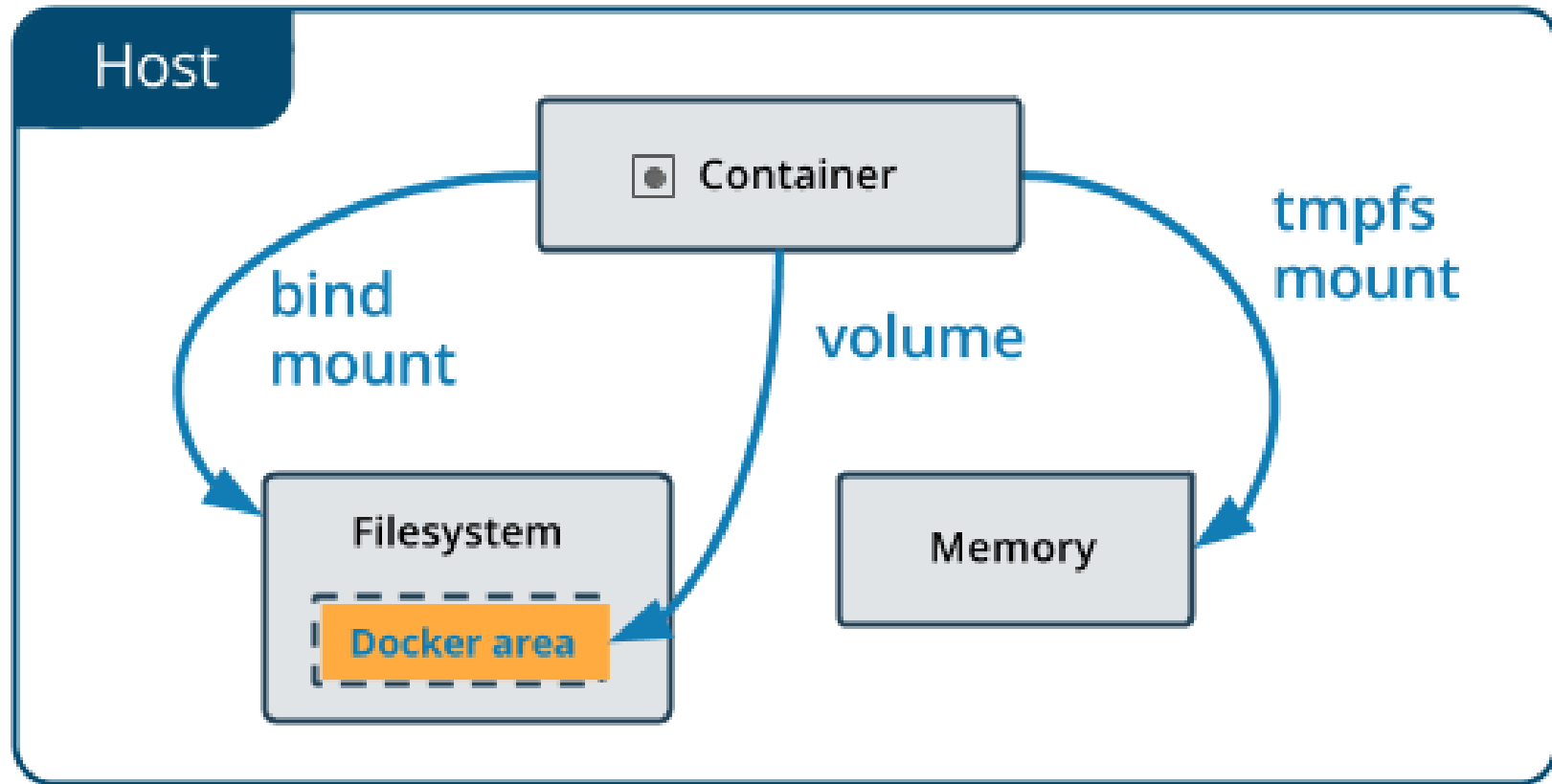
- The **backing filesystem** is the filesystem where **/var/lib/docker/** is located.
- Some **storage drivers only** work with **specific backing filesystems**.

Storage Driver	Supported Backing Filesystem
overlay2, overlay	xfs, ext4
aufs	xfs, ext4
devicemapper	direct-lvm
btrfs	btrfs
zfs	zfs
vfs	any filesystem

```
root@host:~# docker info
Server:
  Containers: 1
  Images: 4
  Server Version: 20.10.7
  Storage Driver: overlay2
    Backing Filesystem: extfs
    Supports d_type: true
    Native Overlay Diff: true
    userxattr: false
```

1.Volumes

1.Volumes



1.Volumes

- **Volumes** are preferred for **persisting** data generated by & used by **Docker containers**.
- Volumes have several **advantages** over bind mounts:
 - Volumes are easier to back up or migrate than bind mounts.
 - You can manage volumes using Docker CLI commands or the Docker API.
 - Volumes work on both Linux & Windows containers.
 - Volumes can be more safely shared among multiple containers.
 - Volume drivers let you store volumes on remote hosts or cloud providers, to encrypt the contents of volumes, or to add other functionality.
 - New volumes can have their content pre-populated by a container.

```
root@host:~# docker volume create myvolume  
myvolume
```

Create & Manage Volumes

- ❑ list volumes by **docker volume ls** ,
- ❑ remove volumes by **docker volume rm** :

```
root@host:~# docker volume ls
DRIVER      VOLUME NAME
local       myvolume
local       myvolume2

root@host:~# docker volume rm myvolume2
myvolume2

root@host:~# docker volume inspect myvolume
[
  {
    "CreatedAt": "2021-02-10T20:13:02Z",
    "Driver": "local",
    "Labels": {},
    "Mountpoint":
"/var/lib/docker/volumes/myvolume/_data",
    "Name": "myvolume",
    "Options": {},
    "Scope": "local"
  }
]
```

Start a Container with a Volume

❑ If a **container** start with a **volume does not exist**, Docker **will create** volume.

❑ **docker run** has two options: **-v** , **--volume** or **--mount** .

```
root@host:~# docker run -d --name test --mount source=myvol,target=/app nginx
```

```
root@host:~# docker run -d --name test --volume myvol2:/app nginx
```

```
root@host:~# docker inspect test | less
```

```
"Mounts": [  
  {  
    "Type": "volume",  
    "Name": "myvol2",  
    "Source": "/var/lib/docker/volumes/myvol2/_data",  
    "Destination": "/app",  
    "Driver": "local",  
    "Mode": "",  
    "RW": true,  
    "Propagation": ""  
  },  
]
```

Removing a Running Volume

- ❑ **docker inspect** shows **correct source & destination** which it is R/W.
- ❑ First **stop the container** & then **remove** the volume.

```
root@host:~# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	PORTS	NAMES
Acc7b60711de	nginx	"/docker-entrypoint.sh"	13 minutes ago	80/tcp	test

```
root@host:~# docker container stop test
test
```

```
root@host:~# docker container rm test
test
```

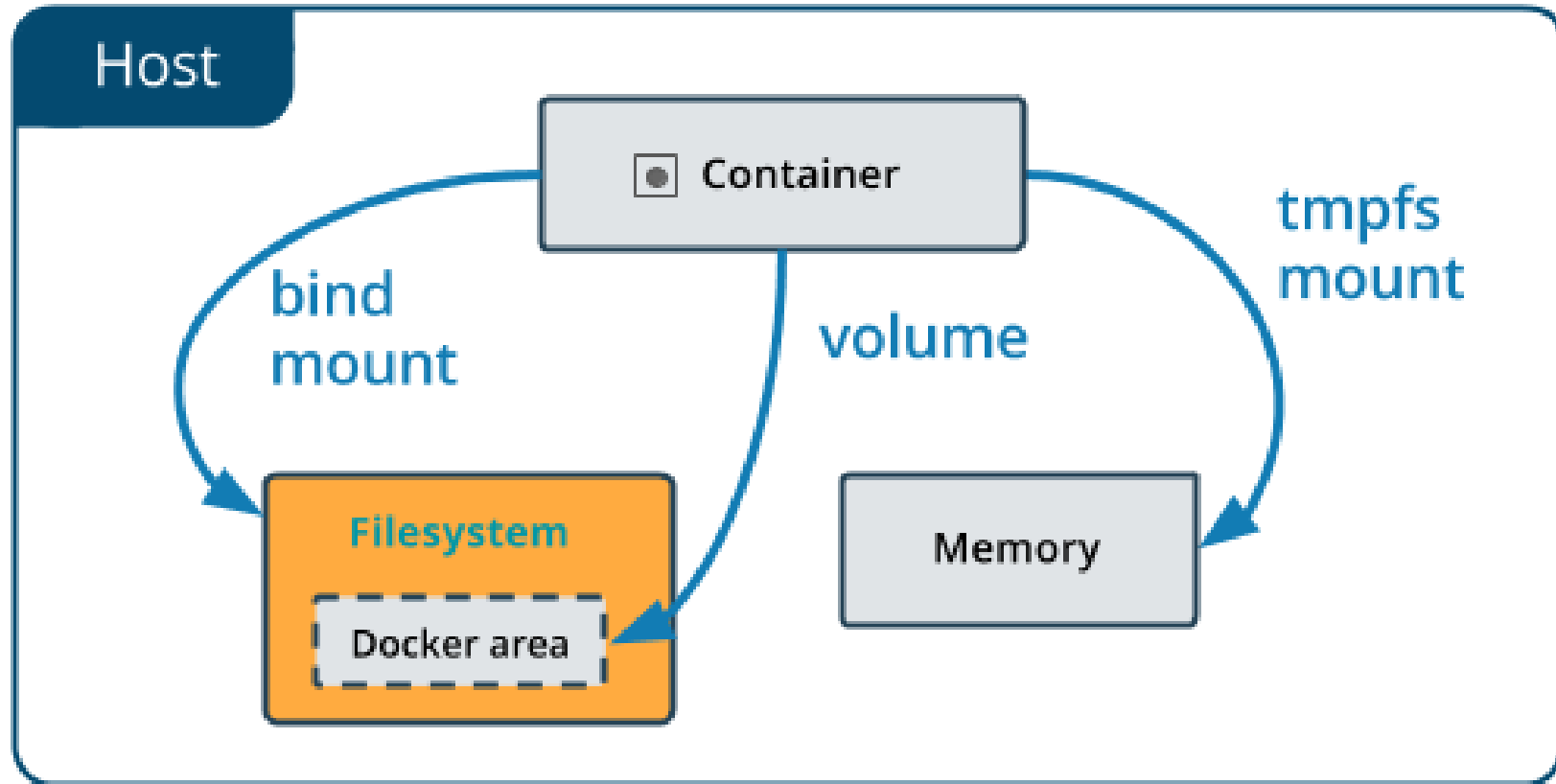
```
root@host:~# docker volume rm myvol2
myvol2
```

```
root@host:~# docker volume ls
```

DRIVER	VOLUME NAME
local	myvol

2.Bind Mounts

2. Bind Mounts



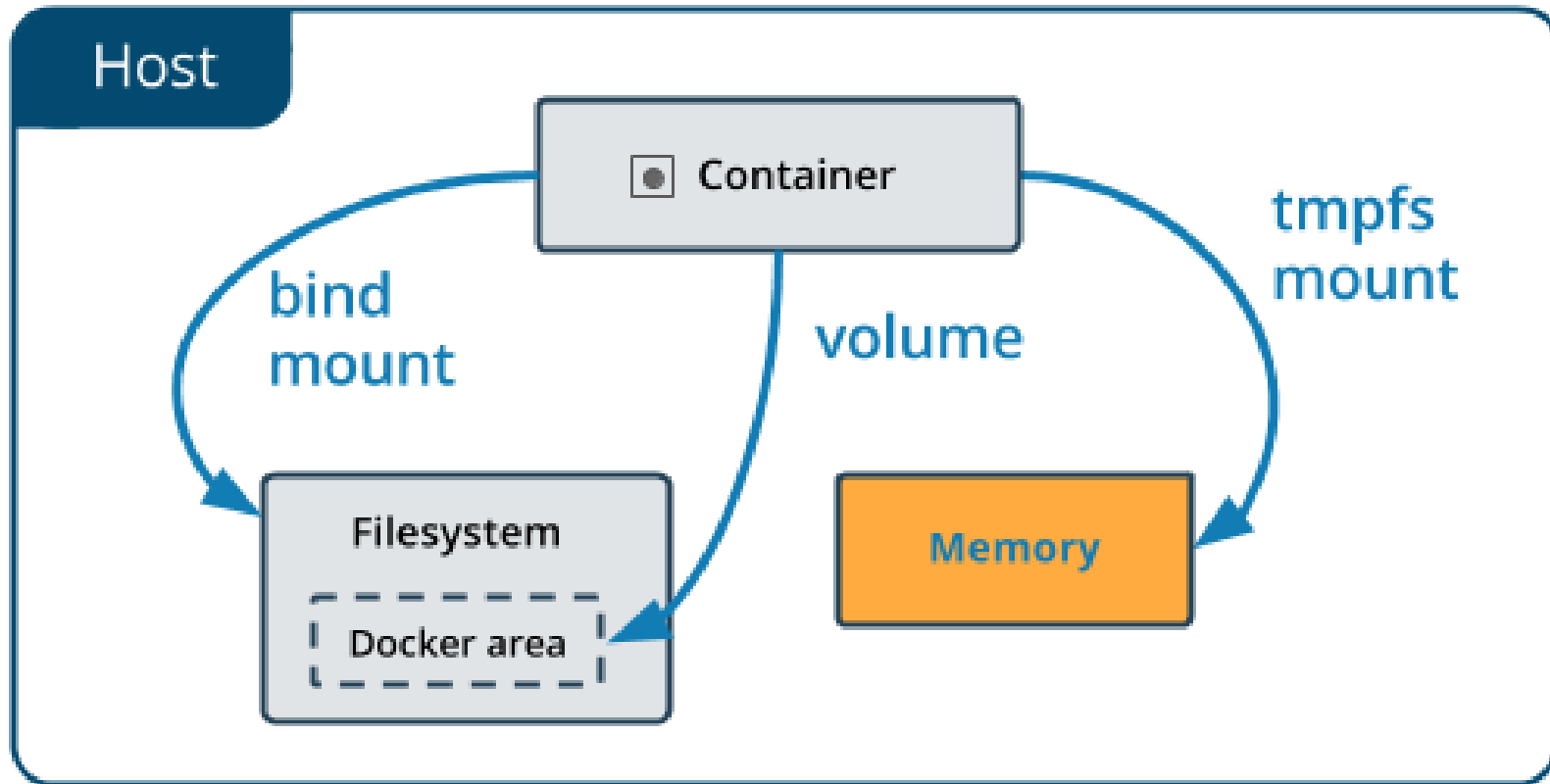
Create & Manage Bind Mounts

- When **bind mount** used, a **file or directory** on **host** machine **mounted into a container**.
- By contrast, when **volume** used, a **new directory** created within **Docker's storage directory** on **host** machine & Docker manages that directory's contents.
- For managing bind mounts, **Docker CLI commands can not used**.

```
root@host:~# docker run -dit --name test --mount type=bind,source=/home/dir,target=/app nginx
root@host:~# docker run -dit --name test --volume /home/dir:/app nginx
root@host:~# docker inspect test | less
  "Type": "bind",
  "Source": "/home/dir",
  "Destination": "/app",
  "Mode": "",
  "RW": true,
  "Propagation": "rprivate"
```

3.Tmpfs Mount

3. Tmpfs Mount



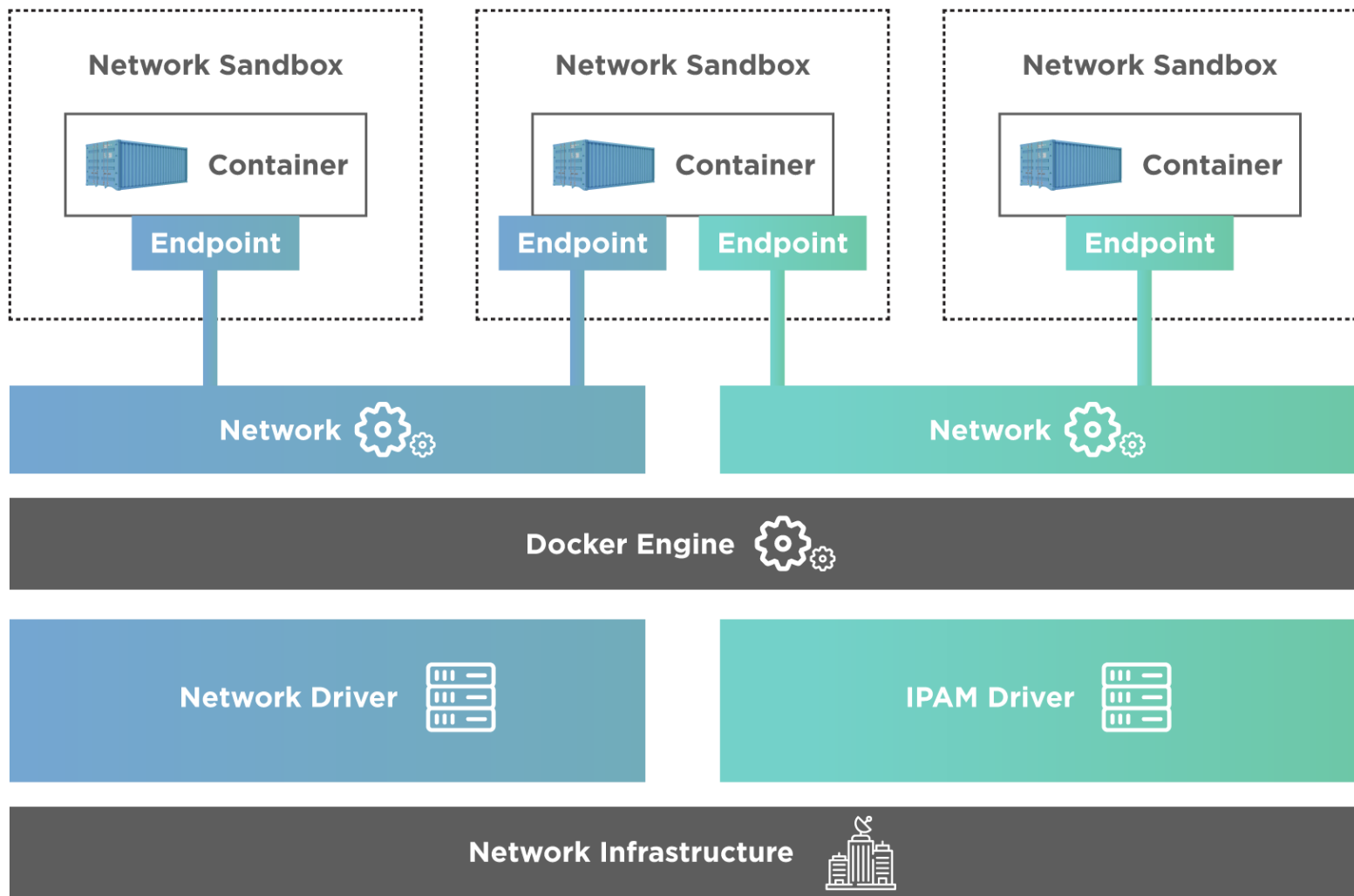
Create & Manage Tmpfs Mounts

- **Volumes & bind mounts** let **share files** between **host & container** so data can persist.
- In **tmpfs mount**, container can **create files outside** the container's **writable layer**.
- A tmpfs mount is **temporary** & only persisted in **host memory**.
- When container **stops**, tmpfs mount is **removed** & files written there **won't be persisted**.

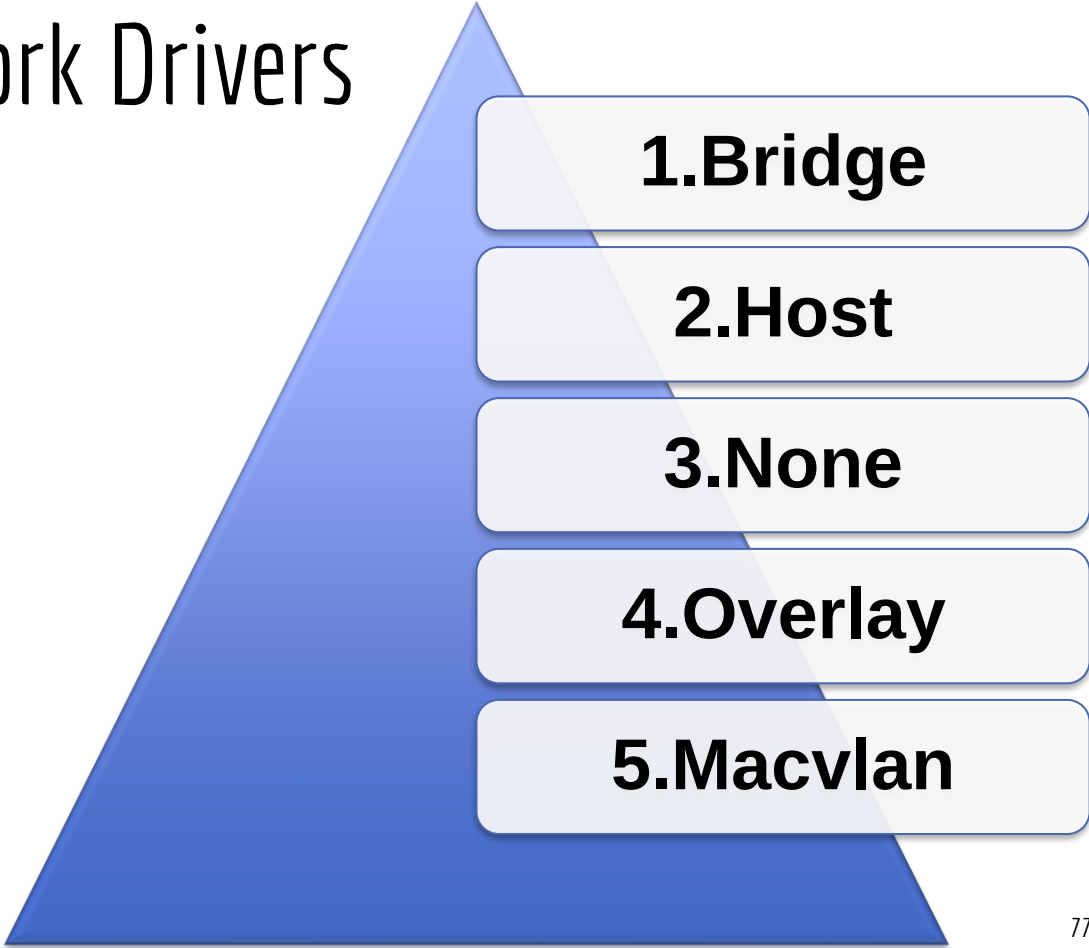
```
root@host:~# docker run -dit --name test --mount type=tmpfs,destination=/app nginx
root@host:~# docker run -dit --name test --tmpfs /app nginx
root@host:~# docker container inspect test
  "Type": "tmpfs",
  "Source": "",
  "Destination": "/app",
  "Mode": "",
  "RW": true,
  "Propagation": ""
```

Docker Networks

Container Network Model (CNM)



Docker Native Network Drivers



1.Bridge

2.Host

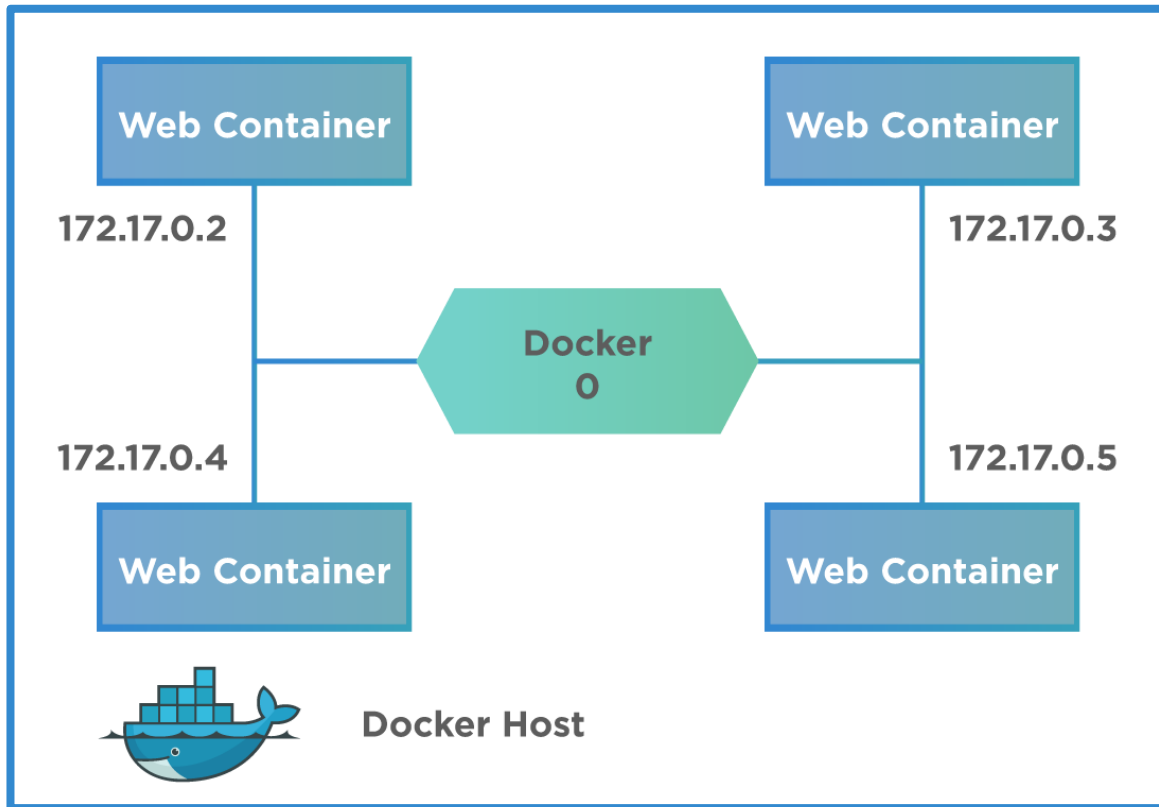
3.None

4.Overlay

5.Macvlan

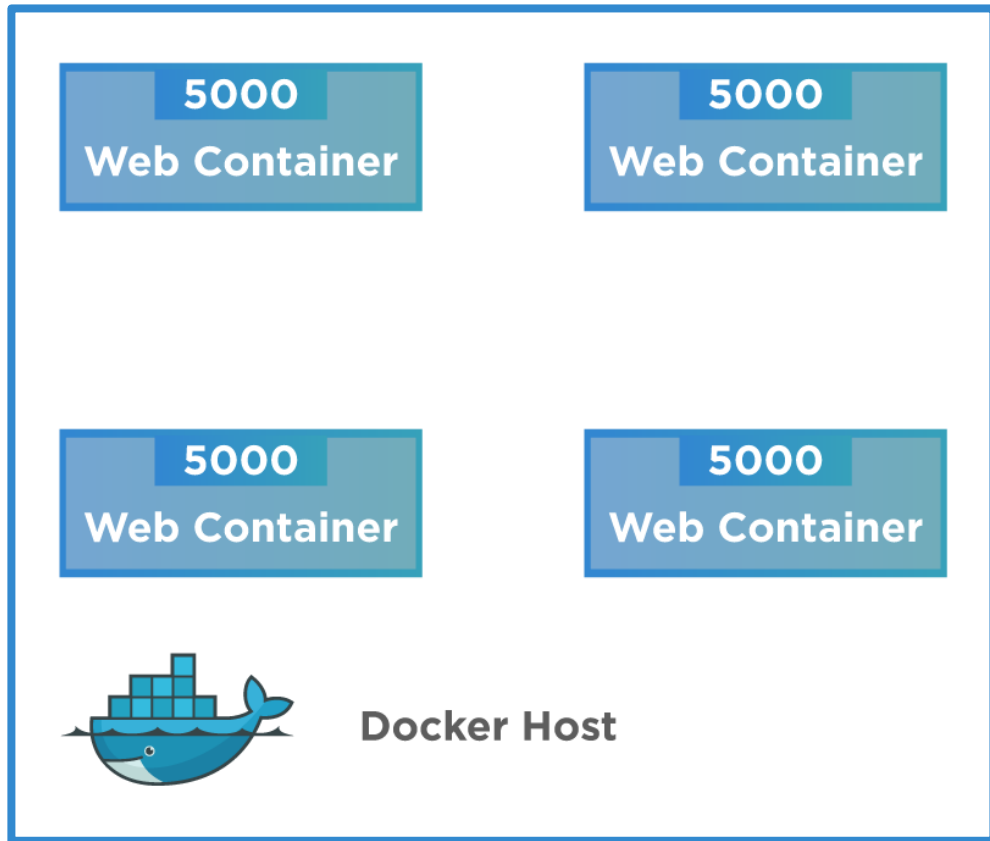
1. Bridge

- **Private default network driver.**
- All containers get an **internal IP**, So they can **access** each other.
- Bridge networks usually used when apps run in **standalone containers** that need to communicate.



2.Host

- It removes network isolation between host & containers to use **host's network directly**.
- So here **cannot** run **multiple web containers** on a **same host & same port** as the port is now common to all containers in the host network.
- Useful for **management container** to run on every **Docker Host**.



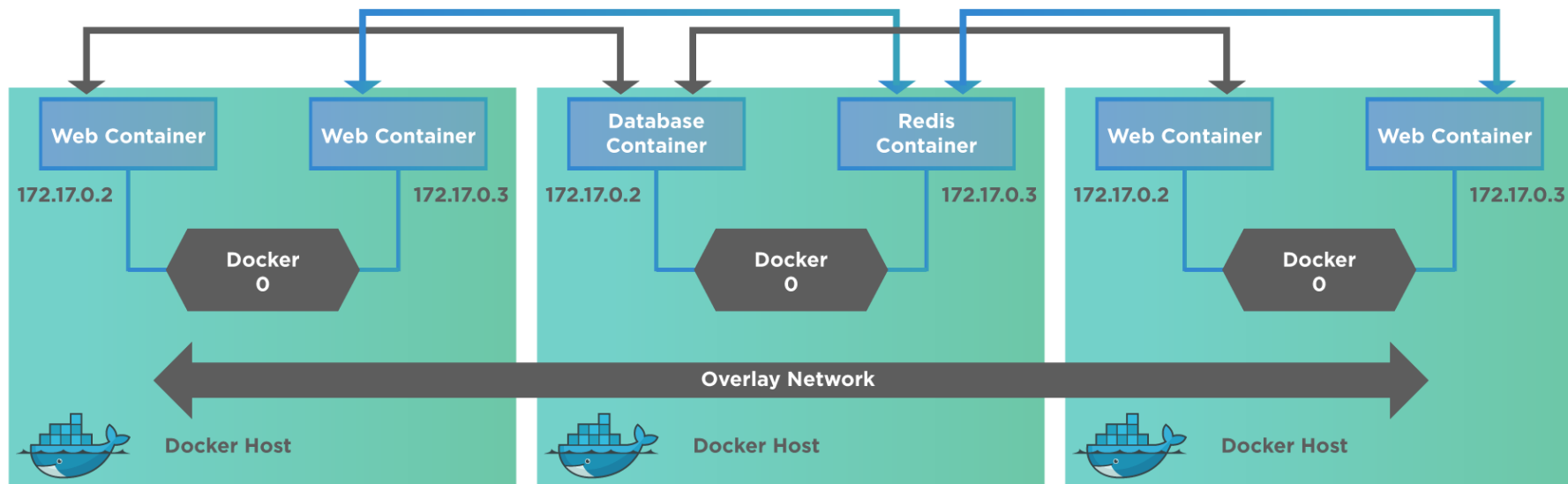
3.None

- For this container, **disable all networking**.
- Usually used when you want to completely **disable networking stack** on a container.
- None is **not available for swarm** services.



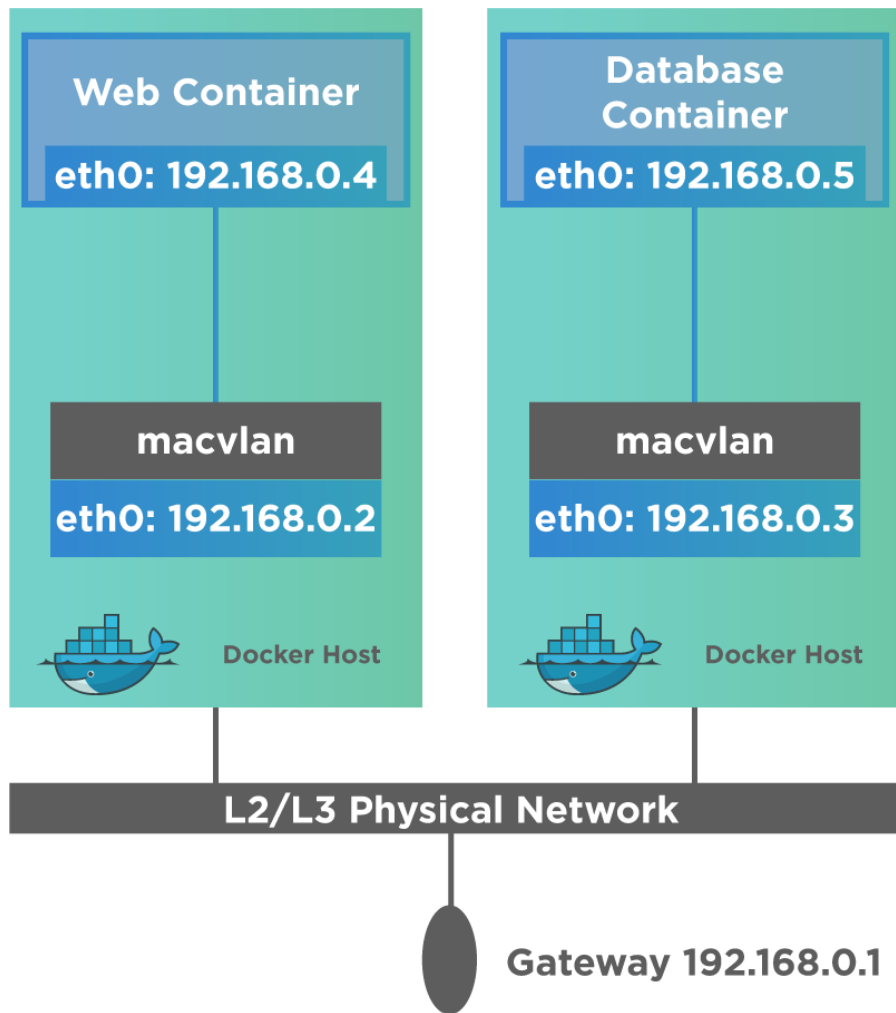
4.Overlay

- Overlay networks connect **multiple Docker daemons** together.
- Only available with **Docker EE & Docker Swarm** enabled.



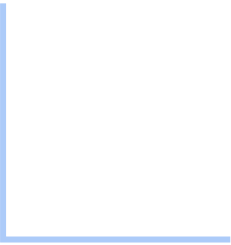
5. Macvlan

- Macvlan networks allow you to **assign a MAC address to a container**, making it appear as a **physical device** on network.
- Docker daemon **routes traffic** to containers by **their MAC addresses**.





Creating a Bridge Network



Creating a Bridge Network

```
root@host:~# docker network create --driver bridge network-app1
F689ab8972b6cee168e678dc3fecc76803cc777cdd82858e15954dad32e8f7dc
```

```
root@host:~# docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
c79ea59cbb31	bridge	bridge	local
0c315edaa154	host	host	local
f689ab8972b6	network-app1	bridge	local
0aeb10a85601	none	null	local

```
root@host:~# ip address show
```

```
2: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    link/ether 02:42:15:be:99:07 brd ff:ff:ff:ff:ff:ff
```

```
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
```

```
7: br-f689ab8972b6: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue
state DOWN
```

```
    link/ether 02:42:2d:f0:52:b7 brd ff:ff:ff:ff:ff:ff
```

```
    inet 172.19.0.1/16 brd 172.19.255.255 scope global br-f689ab8972b6
```

```
root@host:~# docker network inspect c79ea59cbb31
```

Running Containers on a Bridge Network

❑ Now, after **creating a new network**, we're going to **add containers** to it:

```
root@host:~# docker run -dit --name ubuntu1 --network network-app1 ubuntu
09e2bba2c969c4970285163b1b563a9bb4c61703895762b7536dc7f195241eff
root@host:~# docker run -dit --name centos1 --network network-app1 centos
760699752c924136ccc19ad26bb739df1a2c2cdaf6ae365e88e578016af2a390
root@host:~# docker exec -it ubuntu1 /bin/bash
root@09e2bba2c969:/# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.19.0.2 netmask 255.255.0.0 broadcast 172.19.255.255
    ether 02:42:ac:13:00:02 txqueuelen 0 (Ethernet)
root@09e2bba2c969:/# ping 172.19.0.3
PING 172.19.0.3 (172.19.0.3): 56 data bytes
64 bytes from 172.19.0.3: icmp_seq=0 ttl=64 time=0.797 ms
64 bytes from 172.19.0.3: icmp_seq=1 ttl=64 time=0.094 ms
```

Removing a Bridge Network

❏ For **removing** a network driver, just use **docker network remove** :

```
root@host:~# docker stop ubuntu1 centos1
```

```
ubuntu-app1
```

```
centos-app1
```

```
root@host:~# docker network remove network-app1
```

```
network-app1
```

```
root@host:~# docker network ls
```

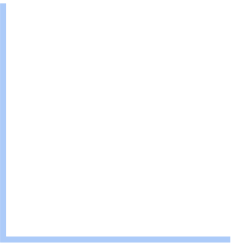
NETWORK ID	NAME	DRIVER	SCOPE
c79ea59cbb31	bridge	bridge	local
0c315edaa154	host	host	local
0aeb10a85601	none	null	local

```
root@host:~# docker network prune
```

```
--> Remove All Unused Networks
```



Running Containers on Host Network



Running Containers on a Host Network

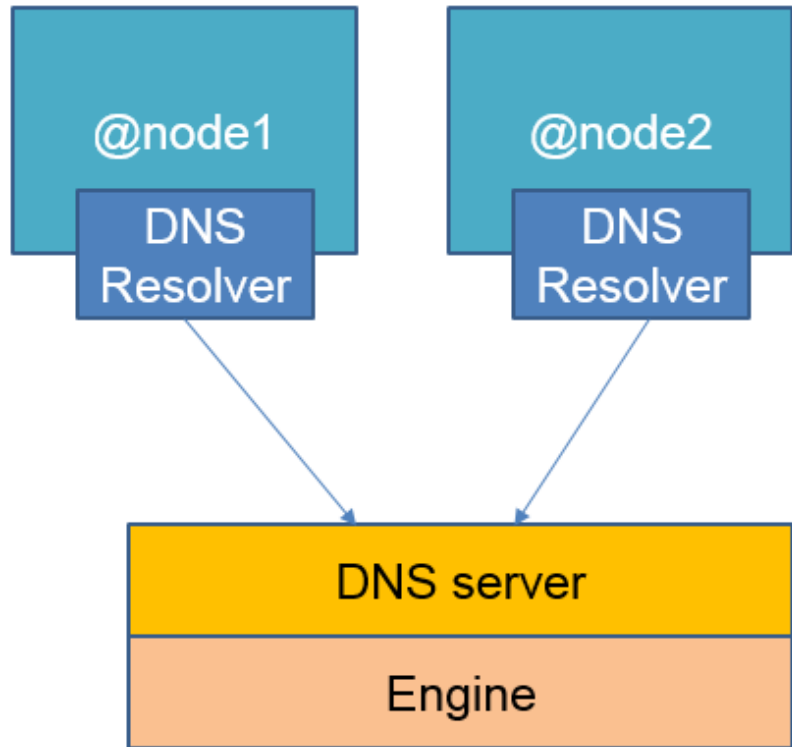
❑ Running a container exactly **mapped on the host** network interface:

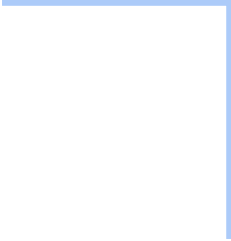
```
root@host:~# docker run -dit --name nginx-host --network host nginx
d8d4905609517c18685fe00f567bc081d6700bf8513708fc8f976c4b49fc93dd
root@host:~# ip address show
2: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    link/ether 02:42:15:be:99:07 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc state UP group
default qlen 1000
    link/ether 00:0c:29:c8:44:df brd ff:ff:ff:ff:ff:ff
    inet 192.168.111.138/24 brd 192.168.111.255 scope global dynamic ens33
root@host:~# elinks http://192.168.111.138
root@host:~# elinks http://172.17.0.1
```

Docker Embedded DNS Server

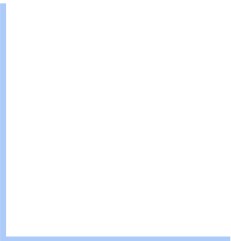
Docker Embedded DNS Server

- Containers also can **reach** each other using their **names** instead of IPs.
- **Docker-Engine** has a **built-in DNS server** that helps containers in a **user-defined bridge network** can resolve each other using **container name**.
- **Built-in DNS server** always runs at address **127.0.0.11**.
- Containers on **default bridge network** can only access each other by IP addresses.





Publishing Ports & Configuring DNS



Publishing Ports of Containers

- **By default**, when a container created, it **doesn't publish** any of its ports to outside.
- To make a port available to **services outside of Docker**, or to containers which **are not connected** to the container's network, use the **--publish** or **-p** flag.
- It'll creates a **firewall rule** which **maps a container port** to a **port on the host**.

Flag Value	Description
-p 8080:80	Map TCP port 80 in container to port 8080 on Docker host.
-p 192.168.1.100:8080:80	Map TCP port 80 in container to port 8080 on Docker host for connections to host IP 192.168.1.100.
-p 8080:80/udp	Map UDP port 80 in container to port 8080 on Docker host.
-p 8080:80/tcp -p 8081:80/udp	Map TCP port 80 in the container to TCP port 8080 on Docker host, & map UDP port 80 in container to UDP port 8081 on Docker host.

Publishing Ports of Containers

❑ Some example of using **--publish** flag. (Publish to a random port with **-P**)

```
root@host:~# docker container run -dit --publish 8080:80 --name web1 nginx  
e6ead9985f27d94acde7c3ff13b1fa6374417ed83ed998ff6523f9ec85a865a0
```

```
root@host:~# docker container run -dit -P --name web2 nginx  
5ea9791996e0508de6ef7d4e78fbbe445e59764a617d242ea4e27fa6db52b977
```

```
root@host:~# docker ps -a
```

CONTAINERID	IMAGE	COMMAND	CREATED	PORTS	NAMES
e6ead998227	nginx	"/docker-entrypoint"	5 minutes	0.0.0.0:8081->80/tcp	web1
5ea97996e20	nginx	"/docker-entrypoint"	2 seconds	0.0.0.0:49153->80/tcp	web2

Configuring DNS for Containers

- By default, **container inherits DNS of host** from `/etc/resolv.conf` file.
- Containers with **default bridge network** get a **copy of this file**,
- Whereas containers with a **custom network** use **Docker's embedded DNS server**, which **forwards** external DNS lookups to host DNS server.

❑ With defining DNS:

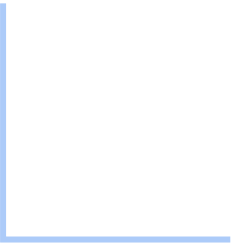
```
root@host:~# docker run -dit --dns 1.1.1.1 --name ubuntu1 ubuntu
root@host:~# docker exec ubuntu1 cat /etc/resolv.conf
nameserver 1.1.1.1
```

❑ Without defining DNS:

```
root@host:~# docker run -dit --name ubuntu2 ubuntu
root@host:~# docker exec ubuntu1 cat /etc/resolv.conf
nameserver 8.8.8.8
nameserver 8.8.4.4
```



Docker Logs & Monitoring



Docker Logs & Events Monitoring

❑ Show **container logs** using **docker logs** & online events using **docker events**:

```
root@host:~# docker logs gitlab-ce
```

```
2021-11-28 18:51:51.528046 TRC start, timeoutMicroS: 5000000 size 2| net/Socket.cpp:327
2021-11-28 18:51:51.528064 TRC Have 1 new children.| wsd/LOOLWSD.cpp:4187
Ready to accept connections on port 9980.
```

```
root@host:~# docker events
```

```
2021-11-28T18:03:53.614817361Z container exec_die e9cfdc61631529a (execID=9847d7c2bd3cef786,
exitCode=0, image=store/gitlab/gitlab-ce:11.10.4-ce.0, name=gitlab)
```

```
root@host:~# docker top nginx3
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1320665	1320644	0	Nov24	pts/0	00:00:00	/bin/bash /assets/wrapper
root	1320745	1320665	0	Nov24	pts/0	00:00:04	runsvdir -P /opt/gitlab/servicelog
root	1320778	1320777	0	Nov24	?	00:00:00	svlogd -tt /var/log/gitlab/ssh
...							

Limit Resource Usage

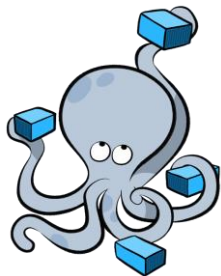
- ❑ By default there is **no restriction** on the resources that containers can use.
- ❑ **A container can end up all of the resources on host machine.**

```
root@host:~# docker run -dit --cpus 1.0 ubuntu          --> Assign 1 CPU to container
root@host:~# docker run -dit --cpu-shares 1024 ubuntu    --> Assign from 1~1024 Cycle
root@host:~# docker run -dit --memory 200m centos        --> Limit memory usage to 200Mb
root@host:~# docker run -dit --memory-reservation 150m --memory 200m centos
                                     --> Set soft limit for memory usage to warn before exceeding limit
```

```
root@host:~# docker stats
```

CONTAINERID	NAME	CPU %	MEM USAGE/LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
Da1b70380bd	centos	0.26%	161.6MiB/200MiB	80.78%	726B/0B	185MB/917MB	11
25631035d56	ubuntu	0.03%	418.6MiB/11.7GiB	3.50%	936B/0B	5.52MB/932MB	11
E9cfdc61615	gitlab	3.21%	3.44GiB/11.69GiB	29.43%	9.27kB/684B	1.06GB/14.8GB	411

Docker Compose

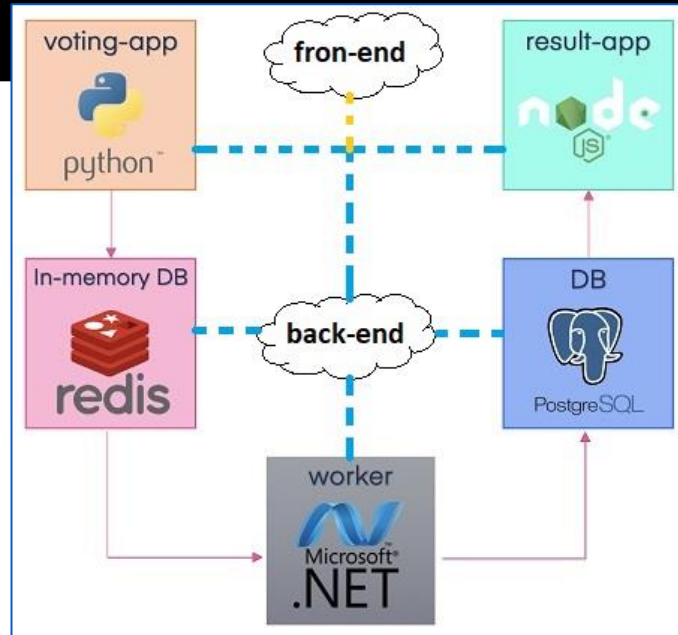


Example: Voting-App

- For this example download **Docker Sample Voting app** which is available in **GitHub**:

```
root@host:~# git clone https://github.com/docker-samples/example-voting-app.git
root@host:~# cd example-voting-app
```

- This app is consist of **several components**.
- This simple app will be used to showcase how easy it is to set up an **entire application stack** consist of **different components** in docker.



Running Containers of Voting App

- In this example needs to make **voting-app** aware of **redis service**. So we add a **link option** to **redis** while running the **voting-app container** :

```
root@host:~# docker run -d --name redis redis:latest
root@host:~# docker run -d --name vote -p 5000:80 --link redis:redis voting-app
root@host:~# docker run -d --name=db -e POSTGRES_PASSWORD=postgres postgres:9.4
root@host:~# docker run -d --name=worker --link db:db --link redis:redis worker
root@host:~# docker run -d --name=result -p 5001:80 --link db:db result-app
```

- Now open webpage of **localhost:5000** in your browser.
- **Caution** : Using links this way is deprecated and the support may be removed in future!

Generate Docker Compose File

```
version: "3.8"
services:
  vote:
    image: voting-app:latest
    command: python app.py
    volumes:
      - ./vote:/app
    ports:
      - "5000:80"
  redis:
    image: redis:alpine
    ports: ["6379"]
  worker:
    image: worker:latest
  db:
    image: postgres:9.4
    environment:
      POSTGRES_USER: "postgres"
      POSTGRES_PASSWORD: "postgres"
  result:
    image: result-app:latest
    command: nodemon server.js
    volumes:
      - ./result:/app
    ports:
      - "5001:80"
      - "5858:5858"
```

Generate Docker Compose File

```
version: "3.8"
services:
  vote:
    build: ./vote
    command: python app.py
    volumes:
      - ./vote:/app
    ports:
      - "5000:80"
  redis:
    image: redis:alpine
    ports: ["6379"]
  worker:
    build: ./worker
  db:
    image: postgres:9.4
    environment:
      POSTGRES_USER: "postgres"
      POSTGRES_PASSWORD: "postgres"
  result:
    build: ./result
    command: nodemon server.js
    volumes:
      - ./result:/app
    ports:
      - "5001:80"
      - "5858:5858"
```

Running Docker Compose

❑ First **stop the container** & then **remove** the volume.

```
root@host:~# docker-compose -f docker-compose.yaml up -d
```

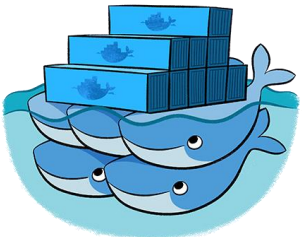
```
root@host:~# docker-compose -f docker-compose.yaml ps
```

Name	Command	Stat	Ports

voting-app-master_db_1	entrypoint.sh	Up	5432/tcp
voting-app-master_redis_1	entrypoint.sh	Up	0.0.0.0:32769->6379/tcp
voting-app-master_result_1	entrypoint.sh	Up	0.0.0.0:5001->80/tcp
voting-app-master_vote_1	python app.py	Up	0.0.0.0:5000->80/tcp
Voting-app-master_worker_1	dotnet src/Work	Up	

```
root@host:~# docker-compose down
```

Docker Swarm



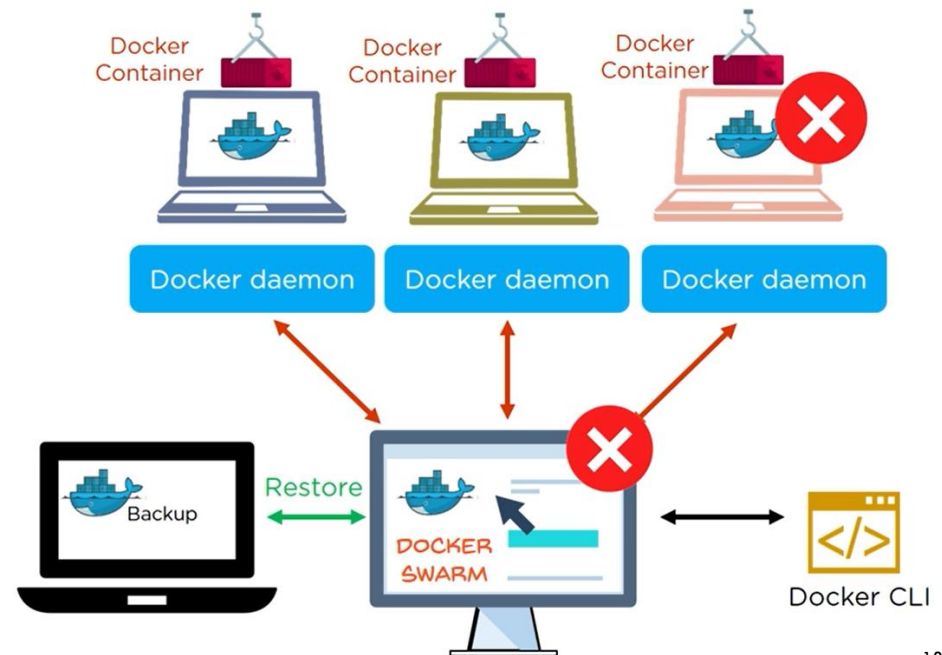
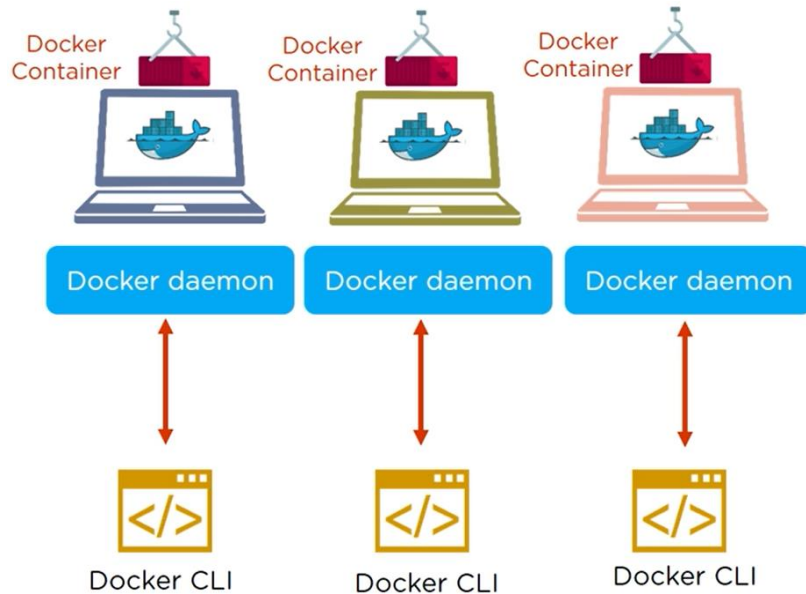
What is Docker Swarm?

- **Docker Swarm** is an **orchestration management tool** that runs on Docker apps.
- It is a service which allows users to **create & manage** a **cluster of Docker nodes** and schedule containers.
- Each node of a Docker Swarm is a **Docker Daemon** which interact using the **Docker API**.
- Here, **services** can be **deployed** and accessed by nodes of a **same cluster**.



What is Docker Swarm?

- In below, if one of the containers **fails**, we can use the **Swarm** to correct that failure.



Features of Docker Swarm

Decentralized Access

High Security

Autoload Balancing

High scalability

Roll-back a task

Docker Swarm Key Concepts

- In Swarm, **containers** launched by **Services**.
- **Service** is a group of containers of the same image.
- **Services** enables to **scale** your application.
- Before you can deploy a service in Swarm, you must have **at least one worker node**.
- There are two types of nodes in Docker Swarm: **Manager, Worker**.

Manager node



Manager node
maintains cluster
management tasks

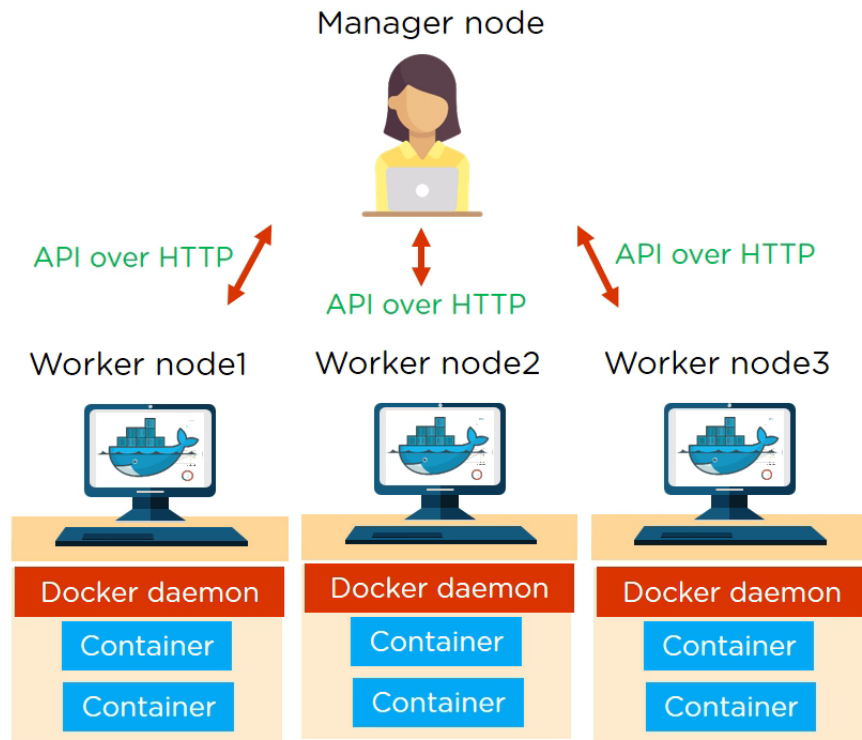
Worker node



Worker nodes receive
and execute tasks from
manager node

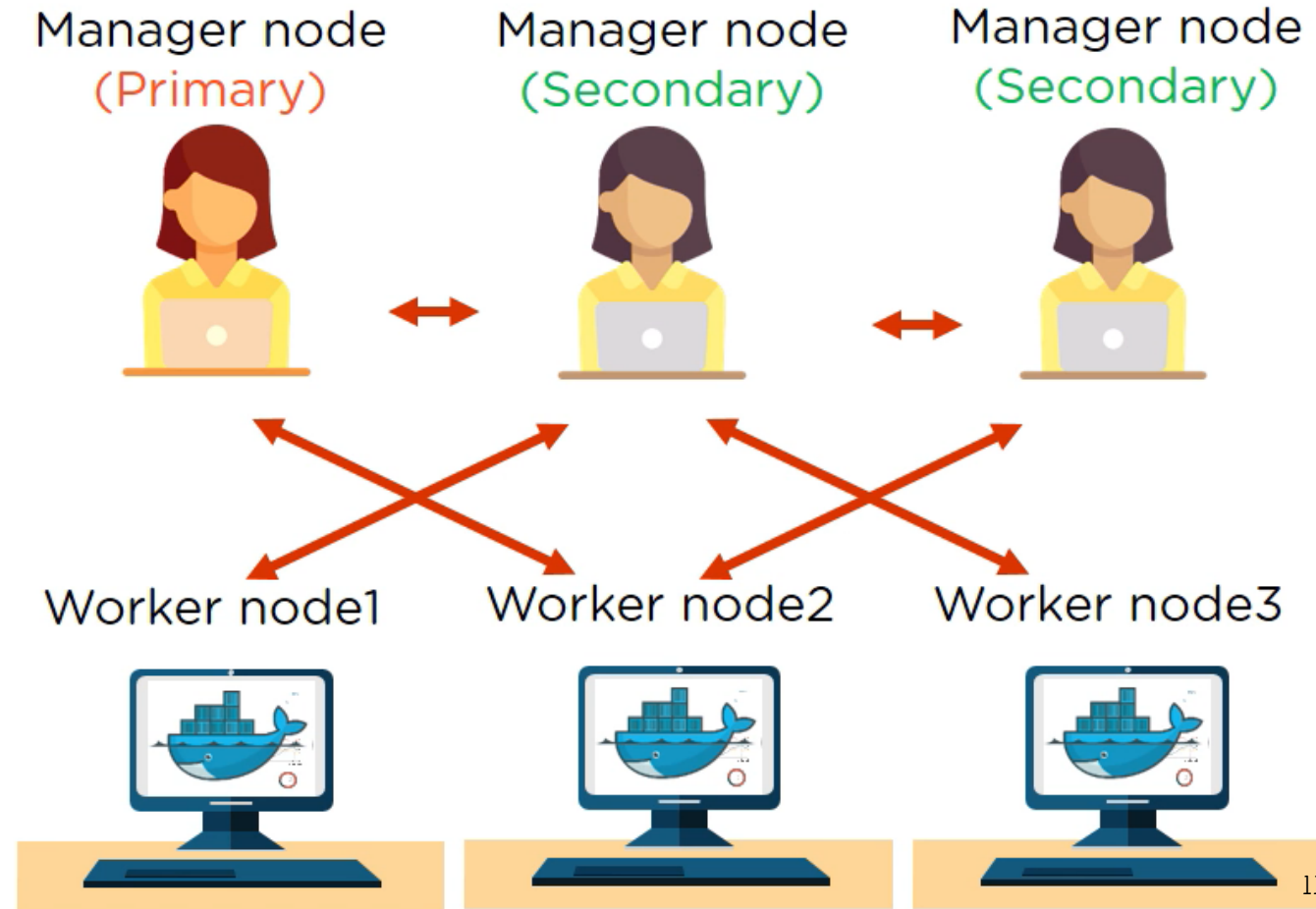
Docker Swarm Key Concepts

- **Manager** node knows the status of all the **Worker** nodes in Swarm cluster.
- **Worker** nodes accept tasks sent from **Manager** node.
- Every **Worker** has an **agent**, which **reports** state of the **node's tasks** to **Manager**.
- **Worker** nodes communicate with **Manager** node using **API over HTTP**.

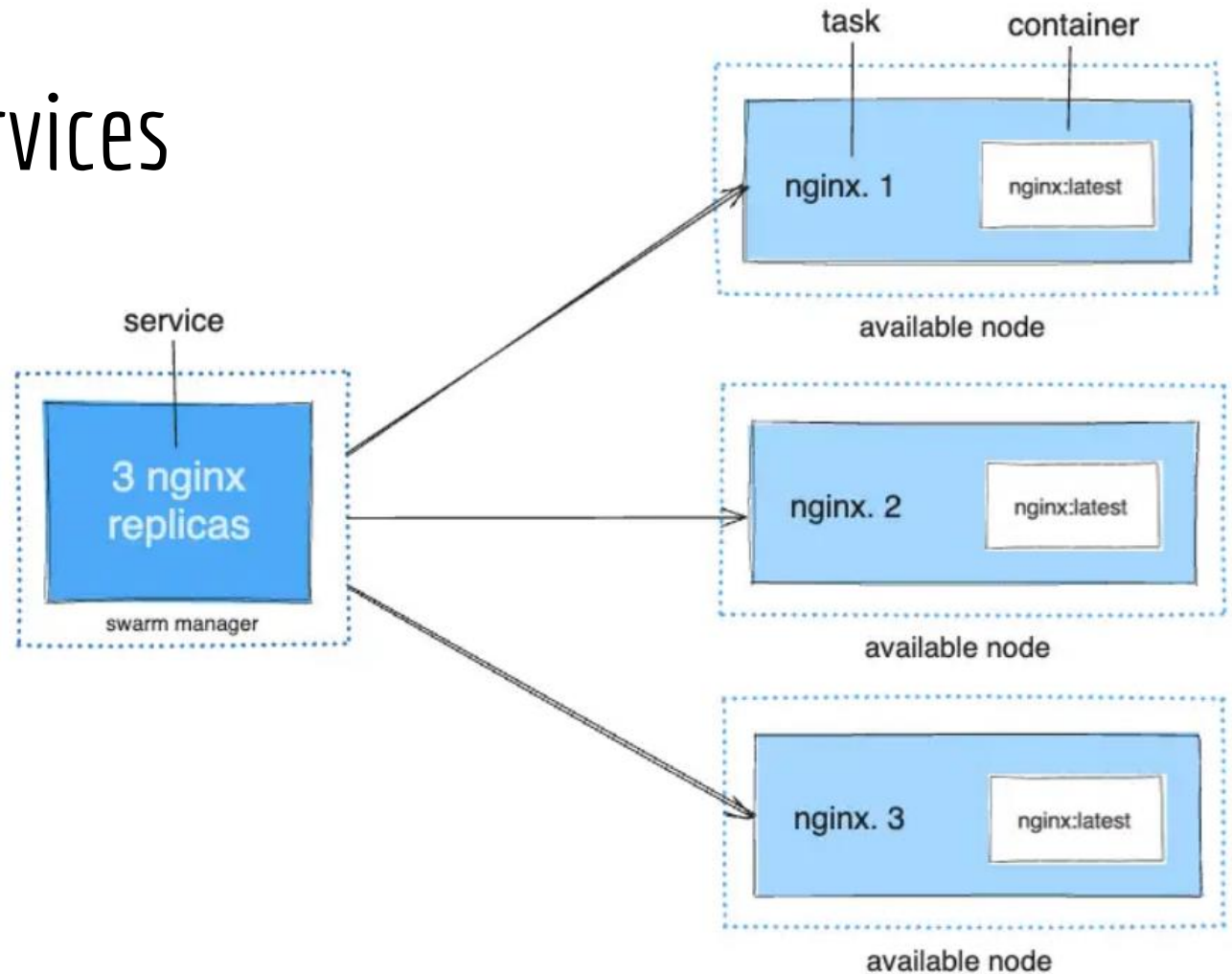


Multiple Managers

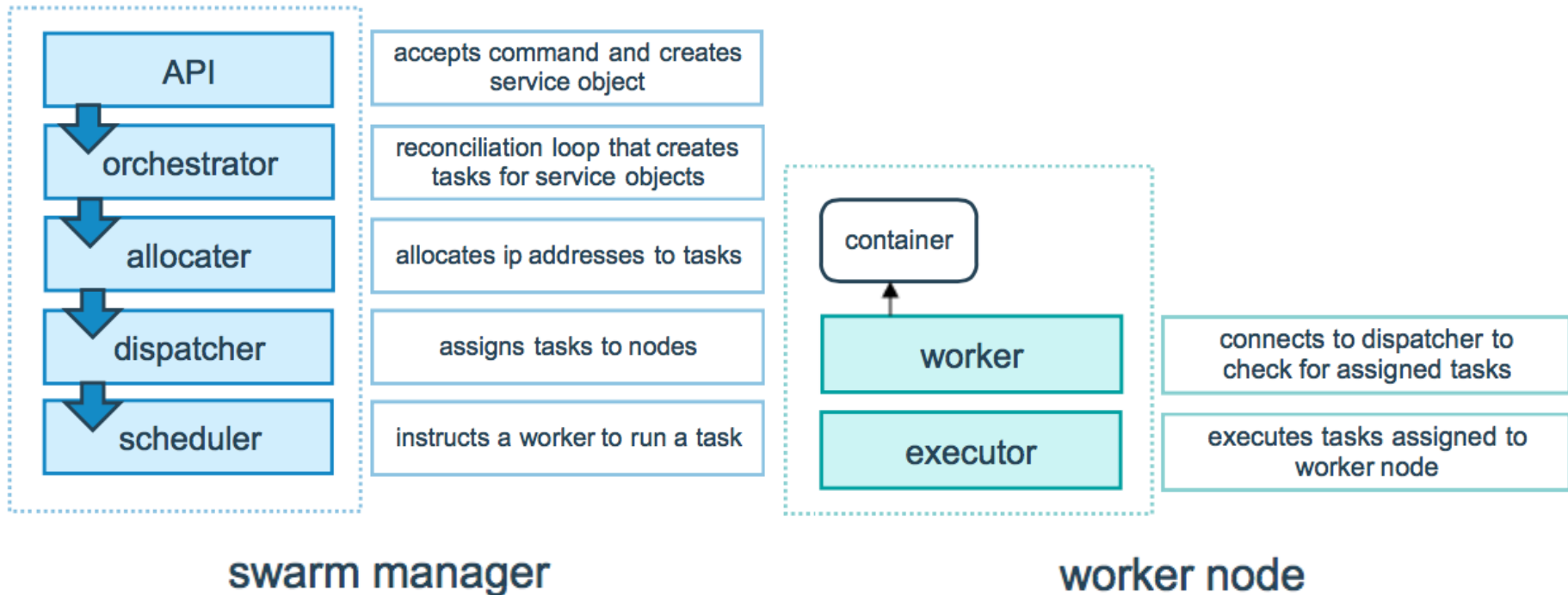
- It is possible to have **multiple manager** nodes on Swarm, but there will be **only one primary manager** node, which gets elected by the other Manager nodes.



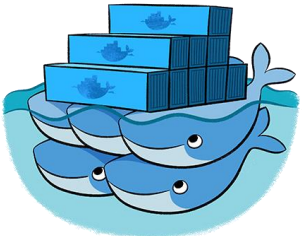
Tasks vs. Services



Tasks & Scheduling



Docker Swarm CLI



Docker Swarm Worker Node

- ❑ Using **docker join** command to join **Worker** nodes into cluster.

```
root@host:~# docker swarm init --advertise-addr 192.168.0.13
```

```
Swarm initialized: current node (cv0vu2txv195aw3cw4mvlatl4) is now a manager.
```

To add a worker to this swarm, run the following command:

```
docker swarm join --token SWMTKN-1-
```

```
5ttj713qief5dfmumellnkp1eklfmztyag1uo2l74g6eq6ivrg-0vkotnzicghfpfsou7no4gtv2  
192.168.10.10:2377
```

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

Docker Swarm Initiation

- ❑ In this scenario, there are two ubuntu hosts, that we want to build a Docker Engine, run Docker commands, and install Docker Swarm.

```
root@host:~# docker swarm join --token SWMTKN-1-  
5ttj713qief5dfmumellnkp1ekl1fmztyag1uo2l74g6eq6ivrg-  
0vkotnzcicghfpfsou7no4gtv2 192.168.10.10:2377
```

This node joined a swarm as a worker.

```
root@host1:~# docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
cv0vumvlatl4 *	node1	Ready	Active	Leader	20.10.0
b9yk1nkc9mnd	node2	Ready	Active		20.10.0

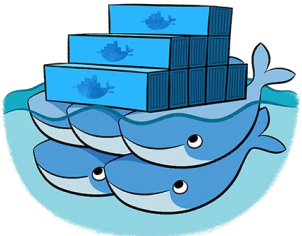
Docker Swarm Service Management

- ❑ In this scenario, there are two ubuntu hosts, that we want to build a Docker Engine, run Docker commands, and install Docker Swarm.

```
root@host:~# docker service create --name mynginx -p 8080:80 --replicas 5
--mount type=bind,source=/tmp,target=/app1 nginx:latest
root@host1:~# docker service ls
root@host1:~# docker service ps mynginx
root@host1:~# docker service inspect mynginx
root@host1:~# docker service logs -f mynginx
root@host1:~# docker service scale mynginx=8
root@host1:~# docker service restart mynginx
root@host1:~# docker service rm mynginx
```

- ✓ Nginx will host web service on all of the cluster nodes on port 8080.

Docker Stack



Docker Stack Deploy

- ❑ Create and update a stack from a compose file on the swarm.

```
root@host:~# docker stack deploy --compose-file docker-compose.yml voting_app
Ignoring unsupported options: links

Creating service voting_app_voting_app
Creating service voting_app_result
Creating service voting_app_voting
Creating service voting_app_worker
Creating service voting_app_redis
Creating service voting_app_postgres
```

Thank You!

Arash Foroughi

2023-24

SEMATEC COMPANY
